AA 273 Midterm

The midterm can be taken at any time during the 24 hour period it is available but must be completed within **a single 5 hour period**. The exam must be submitted no later than 3:00 PM (Pacific) on Saturday, February 7th through Gradescope.

Allowed resources

- Canvas (e.g., homework solutions, lecture notes/videos, starter code, past Ed posts)
- Textbooks
- Built-In Coding Assistants (ie Cursor, Copilot)
- Online documentation for any libraries that have been included in starter code (homework and exam). Examples include Numpy, Scipy, Matplotlib, Open3D, Pytorch, Pandas and standard Python libraries (math, csv).

Resources that are **NOT allowed**

- Material from previous iterations of the class (e.g., homework or exams)
- Discussion with classmates or anyone outside of the class
- The internet (except for documentation for the aforementioned libraries)
- Generative AI tools (except for code editing)
- You are not allowed to post questions relating to the midterm publicly on Ed between May 5 and May 7.

Additional Notes

- We prefer typeset solutions. However, if you choose to handwrite your solutions, the answers must be legible.
- Some questions require coding. Please provide associated code in your submission.
- If there are any questions regarding the exam, please direct them to the CAs through Ed as a private message or through email.

Q1 (20 points)

Controlled Bayes Filter

A discrete state $X_t \in \{1, 2, 3\}$ evolves according to a controlled transition model that depends on a control input $u_t \in \{\mathrm{N}, \mathrm{F}\}$ (No-move vs Forward). The "forward" direction is $1 \to 2 \to 3 \to 1$.

Controlled transition model.

- If $u_t = \mathrm{N}$: $p(x_t = x_{t-1}) = 0.9$, and the remaining 0.1 probability is split equally among the other two states.
- If $u_t = \mathrm{F}$: with probability 0.8 you go forward (i.e., $x_t = x_{t-1} + 1 \bmod 3$), and with probability 0.2 you stay (i.e., $x_t = x_{t-1}$).

Observation model. The observation is $Y_t \in \{1, 2, 3\}$ with

$$p(Y_t = y \mid X_t = x) = \begin{cases} 0.7, & y = x, \\ 0.15, & y \neq x \text{ (each of the other two values).} \end{cases}$$

Initial belief:

$$b_0 = [0.2,\ 0.5,\ 0.3]$$

corresponding to states $[1, 2, 3]$.

Controls and observations:

$$u_1 = \mathrm{F},\ y_1 = 2; \qquad u_2 = \mathrm{N},\ y_2 = 2.$$

(a) **[6 Points]** Write the Bayes filter recursion for

$$b_t(x) = p(X_t = x \mid y_{1:t}, u_{1:t}),$$

including a prediction step using $p(x_t \mid x_{t-1}, u_t)$ and an update step using $p(y_t \mid x_t)$.

(b) **[10 Points]** Compute b_1 and b_2 numerically.

(c) **[4 Points]** Let $u_3 = \mathrm{F}$. Compute the predicted belief

$$\bar{b}_3(x) = p(X_3 = x \mid y_{1:2}, u_{1:3})$$

(i.e., prediction only; no measurement update at time 3).

Controlled Bayes Filter

A discrete state $X_t \in \{1, 2, 3\}$ evolves according to a controlled transition model that depends on a control input $u_t \in \{\mathrm{N}, \mathrm{F}\}$ (No-move vs Forward). The "forward" direction is $1 \to 2 \to 3 \to 1$.

Controlled transition model.

- If $u_t = \mathrm{N}$: $p(x_t = x_{t-1}) = 0.9$, and the remaining 0.1 probability is split equally among the other two states.
- If $u_t = \mathrm{F}$: with probability 0.8 you go forward (i.e., $x_t = x_{t-1} + 1 \bmod 3$), and with probability 0.2 you stay (i.e., $x_t = x_{t-1}$).

Observation model. The observation is $Y_t \in \{1, 2, 3\}$ with

$$p(Y_t = y \mid X_t = x) = \begin{cases} 0.7, & y = x, \\ 0.15, & y \neq x \text{ (each of the other two values).} \end{cases}$$

Initial belief:

$$b_0 = [0.2,\ 0.5,\ 0.3]$$

corresponding to states $[1, 2, 3]$.

Controls and observations:

$$u_1 = \mathrm{F},\ y_1 = 2; \qquad u_2 = \mathrm{N},\ y_2 = 2.$$

(a) **[6 Points]** Write the Bayes filter recursion for

$$b_t(x) = p(X_t = x \mid y_{1:t}, u_{1:t}),$$

including a prediction step using $p(x_t \mid x_{t-1}, u_t)$ and an update step using $p(y_t \mid x_t)$.

(b) **[10 Points]** Compute b_1 and b_2 numerically.

(c) **[4 Points]** Let $u_3 = \mathrm{F}$. Compute the predicted belief

$$\bar{b}_3(x) = p(X_3 = x \mid y_{1:2}, u_{1:3})$$

(i.e., prediction only; no measurement update at time 3).

a)

$$p(x_t \mid y_{1:t-1}, u_t) = \int_{x_{t-1}} p(x_t \mid x_{t-1}, u_t)\, p(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}$$

$$p(x_t \mid y_{1:t}) = \frac{p(y_t \mid x_t)\, p(x_t \mid y_{1:t-1})}{\int_{x_t} p(y_t, x_t)\, p(x_t \mid y_{1:t-1})\, dx_t}$$

b)

$$T_N(x_t \mid x_{t-1}, u_t) = \begin{bmatrix} 0.2 & 0.8 & 0 \\ 0 & 0.2 & 0.8 \\ 0.8 & 0 & 0.2 \end{bmatrix} \qquad T_F(x_t \mid x_{t-1}, u_t) = \begin{bmatrix} 0.9 & 0.05 & 0.05 \\ 0.05 & 0.9 & 0.05 \\ 0.05 & 0.05 & 0.9 \end{bmatrix}$$

$$p(x_t \mid y_0) = p(x_t) = b_0$$

$$p(\bar{b}_1 \mid b_0, u_F) = \begin{bmatrix} 0.2 & 0.8 & 0 \\ 0 & 0.2 & 0.8 \\ 0.8 & 0 & 0.2 \end{bmatrix} \begin{bmatrix} 0.2 \\ 0.5 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.44 \\ 0.34 \\ 0.22 \end{bmatrix}$$

$$p(b_1 \mid y_1 = 2) = \frac{p(y_1 \mid x_1)\, p(x_1 \mid y_0)}{p(y_1 \mid x=1)\, p(x=1) + p(y_1 \mid x=2)\, p(x=2) + p(y_1 \mid x=3)\, p(x=3)}$$

$$\frac{p(y=2|x_1)\cdot \bar{b}_1}{\int p(y=2|x_1)\cdot \bar{b}_1 \, dx_{t-1}}$$

$$p(y=2|x_t) = \begin{bmatrix} 0.15 \\ 0.7 \\ 0.15 \end{bmatrix}$$

$$\frac{\begin{bmatrix} 0.15 \\ 0.7 \\ 0.15 \end{bmatrix} \times \begin{bmatrix} 0.44 \\ 0.34 \\ 0.22 \end{bmatrix}}{\text{dot}\left(\begin{bmatrix} 0.15 \\ 0.7 \\ 0.15 \end{bmatrix} \cdot \begin{bmatrix} 0.44 \\ 0.34 \\ 0.22 \end{bmatrix}\right)}$$

$$b_1 = \begin{bmatrix} 0.1958 \\ 0.7067 \\ 0.0979 \end{bmatrix}$$

$$p(\bar{b}_2|b_1, u_N) = \begin{bmatrix} 0.9 & 0.05 & 0.05 \\ 0.05 & 0.9 & 0.05 \\ 0.05 & 0.5 & 0.9 \end{bmatrix} \begin{bmatrix} 0.1958 \\ 0.7067 \\ 0.0979 \end{bmatrix} = \begin{bmatrix} 0.2165 \\ 0.6503 \\ 0.1332 \end{bmatrix}$$

$$p(b_2|y_2=2) = \frac{p(y_2|x_2)\,p(x_2|y_1)}{p(y_1|x=1)\,p(x=1) + p(y_1|x=2)\,p(x=2) + p(y_1|x=3)\,p(x=3)}$$

$$\frac{p(y=2|x_1)\cdot \bar{b}_1}{\int p(y=2|x_1)\cdot \bar{b}_1 \, dx_{t-1}}$$

$$p(y=2|x_t) = \begin{bmatrix} 0.15 \\ 0.7 \\ 0.15 \end{bmatrix}$$

$$\frac{\begin{bmatrix} 0.15 \\ 0.7 \\ 0.15 \end{bmatrix} \times \begin{bmatrix} 0.2165 \\ 0.6503 \\ 0.1332 \end{bmatrix}}{\text{dot}\left(\begin{bmatrix} 0.15 \\ 0.7 \\ 0.15 \end{bmatrix} \cdot \begin{bmatrix} 0.2165 \\ 0.6503 \\ 0.1332 \end{bmatrix}\right)}$$

$$b_2 = \begin{bmatrix} 0.0640 \\ 0.8967 \\ 0.0394 \end{bmatrix}$$

c)

$$p(\bar{b}_2 | b_1, u_N) = \begin{bmatrix} 0.2 & 0.8 & 0 \\ 0 & 0.2 & 0.8 \\ 0.8 & 0 & 0.2 \end{bmatrix} \begin{bmatrix} 0.0640 \\ 0.8967 \\ 0.0394 \end{bmatrix}$$

$$\bar{b}_3 = \begin{bmatrix} 0.7301 \\ 0.2108 \\ 0.0590 \end{bmatrix}$$

Q2 (20 points)

Quadratic Expectations and the Trace Trick

Let $X \in \mathbb{R}^n$ be a random vector with mean $\mu = \mathbb{E}[X]$ and covariance $\Sigma = \mathbb{E}[(X-\mu)(X-\mu)^\mathsf{T}]$. Let $Q \in \mathbb{R}^{n\times n}$ be a **symmetric** matrix, i.e. $Q = Q^\mathsf{T}$, and let $b \in \mathbb{R}^n$ be a constant vector.

Hint: For any vectors u, v and matrix A of compatible sizes,

$$u^\mathsf{T} A v = \mathrm{tr}\big(A v u^\mathsf{T}\big)\,.$$

(a) **[10 Points]** Derive a closed-form expression for $\mathbb{E}[\|QX+b\|_2^2]$ in terms of (μ, Σ, Q, b). Your final answer should separate a "mean" term and a "covariance" term. **Hint:** $\Sigma = \mathbb{E}[XX^\mathsf{T}] - \mu\mu^\mathsf{T}$.

where the first term is the "mean" term and the second is the "variance"

(b) **[7 Points]** Using the hint above, show that

$$\mathbb{E}\big[(X-\mu)^\mathsf{T} Q (X-\mu)\big] = \mathrm{tr}(Q\Sigma).$$

(c) **[3 Points]** If in addition $Q \succeq 0$ (Q is PSD), give a one-line argument why $\mathbb{E}[(X-\mu)^\mathsf{T} Q (X-\mu)] \geq 0$.

Q2 (20 points)

Quadratic Expectations and the Trace Trick

Let $X \in \mathbb{R}^n$ be a random vector with mean $\mu = \mathbb{E}[X]$ and covariance $\Sigma = \mathbb{E}[(X-\mu)(X-\mu)^\top]$. Let $Q \in \mathbb{R}^{n \times n}$ be a **symmetric** matrix, i.e. $Q = Q^\top$, and let $b \in \mathbb{R}^n$ be a constant vector.

Hint: For any vectors u, v and matrix A of compatible sizes,

$$u^\top A v = \mathrm{tr}\left(A v u^\top\right).$$

(a) **[10 Points]** Derive a closed-form expression for $\mathbb{E}[\|QX+b\|_2^2]$ in terms of (μ, Σ, Q, b). Your final answer should separate a "mean" term and a "covariance" term. **Hint:** $\Sigma = \mathbb{E}[XX^\top] - \mu\mu^\top$.
where the first term is the "mean" term and the second is the "variance"

(b) **[7 Points]** Using the hint above, show that

$$\mathbb{E}\left[(X-\mu)^\top Q (X-\mu)\right] = \mathrm{tr}(Q\Sigma).$$

(c) **[3 Points]** If in addition $Q \succeq 0$ (Q is PSD), give a one-line argument why $\mathbb{E}[(X-\mu)^\top Q(X-\mu)] \geq 0$.

$$E\left[\|QX+b\|_2^2\right]$$

$$= E\left[(QX+b)^T(QX+b)\right]$$

$$= E\left[x^T Q^T Q x + b^T Q x + x^T Q^T b + b^T b\right]$$

$$= E\left[x^T Q^T Q x + b^T Q x + x^T Q^T b + b^T b\right]$$

$$= E\left[x^T Q Q^T x\right] + b^T E[Qx] + E[x^T Q] b + b^T b$$

$$= x^T Q Q x + tr(QQ^T) +$$

Q3 (20 points)

Sensor Fusion and Error Probability

Consider an autonomous vehicle in $\mathbb{R}^2$. Assume the prior over the robot pose state X is Gaussian:

$$X \sim \mathcal{N}(\mu_0, \Sigma_0).$$

At some arbitrary time we are given two measurements Y_1 and Y_2 that are conditionally independent given X, with Gaussian measurement models:

$$(Y_1 \mid X = x) \sim \mathcal{N}(x, \Sigma_1), \qquad (Y_2 \mid X = x) \sim \mathcal{N}(x, \Sigma_2).$$

(a) **[5 Points]** You will fuse these two measurements. Using Bayes' rule, first compute the fused posterior $p(x \mid y_1, y_2) = \mathcal{N}(\mu_f, \Sigma_f)$, **and then** state how the magnitude of Σ_f compares to that of the individual measurement covariances. You may work with the unnormalized posterior distribution and ignore constants that do not depend on x, because the Σ_f which you will compute normalizes the Gaussian via the normalization constant η. g

(b) **[5 Points]** Now let's make our system more efficient by replacing measurements Y_1 and Y_2 with a single sensor that outputs a scalar linear Gaussian measurement W with measurement model

$$W = C^\top X + V$$

where $C \in \mathbb{R}^n$ is a vector of constants and $V \in \mathbb{R}$ is noise, independent of X, and distributed as

$$V \sim \mathcal{N}(0, R).$$

Find $\mathbb{E}[W]$ and $\mathrm{Cov}(W)$, and identify the distribution of W.

(c) **[5 Points]** We would like to know the probability that the measurement W lies within ε of the expected value of that measurement. Find

$$\mathbb{P}(|W - \mathbb{E}[W]| \leq \varepsilon)\,.$$

Your answer should be in terms of the standard normal cumulative distribution function (CDF), defined for a scalar standard normal random variable Z as $\Phi(a) = Pr(Z \leq a) = \int_{-\infty}^{a} \mathcal{N}(0,1)\, dz$.
Hint: $\mathbb{P}(a \leq Z \leq b) = \Phi(b) - \Phi(a)$ for $Z \sim \mathcal{N}(0,1)$.

(d) **[5 Points]** Now suppose we swap the sensor and the new measurement model for W is:

$$W = \cos(X).$$

You are thinking about applying a Kalman filter to the task of estimating the mean and covariance for W over time. Will the KF work? Why or why not?

Q3 (20 points)

Sensor Fusion and Error Probability

Consider an autonomous vehicle in $\mathbb{R}^2$. Assume the prior over the robot pose state X is Gaussian:

$$X \sim \mathcal{N}(\mu_0, \Sigma_0).$$

At some arbitrary time we are given two measurements Y_1 and Y_2 that are conditionally independent given X, with Gaussian measurement models:

$$(Y_1 \mid X = x) \sim \mathcal{N}(x, \Sigma_1), \qquad (Y_2 \mid X = x) \sim \mathcal{N}(x, \Sigma_2).$$

(a) **[5 Points]** You will fuse these two measurements. Using Bayes' rule, first compute the fused posterior $p(x \mid y_1, y_2) = \mathcal{N}(\mu_f, \Sigma_f)$, **and then** state how the magnitude of Σ_f compares to that of the individual measurement covariances. You may work with the unnormalized posterior distribution and ignore constants that do not depend on x, because the Σ_f which you will compute normalizes the Gaussian via the normalization constant η. g

(b) **[5 Points]** Now let's make our system more efficient by replacing measurements Y_1 and Y_2 with a single sensor that outputs a scalar linear Gaussian measurement W with measurement model

$$W = C^\top X + V$$

where $C \in \mathbb{R}^n$ is a vector of constants and $V \in \mathbb{R}$ is noise, independent of X, and distributed as

$$V \sim \mathcal{N}(0, R).$$

Find $\mathbb{E}[W]$ and $\mathrm{Cov}(W)$, and identify the distribution of W.

(c) **[5 Points]** We would like to know the probability that the measurement W lies within ε of the expected value of that measurement. Find

$$\mathbb{P}(|W - \mathbb{E}[W]| \leq \varepsilon).$$

Your answer should be in terms of the standard normal cumulative distribution function (CDF), defined for a scalar standard normal random variable Z as $\Phi(a) = Pr(Z \leq a) = \int_{-\infty}^{a} \mathcal{N}(0,1)\, dz$.
Hint: $\mathbb{P}(a \leq Z \leq b) = \Phi(b) - \Phi(a)$ for $Z \sim \mathcal{N}(0,1)$.

(d) **[5 Points]** Now suppose we swap the sensor and the new measurement model for W is:

$$W = \cos(X).$$

You are thinking about applying a Kalman filter to the task of estimating the mean and covariance for W over time. Will the KF work? Why or why not?

a)

$$p(y_1, y_2) = \int p(y_1, y_2 \mid x)\, p(x)\, dx$$

$$p(x \mid y_1, y_2) = \frac{p(y_1, y_2 \mid x)\, p(x)}{p(y_1, y_2)}$$

$$= \frac{p(y_1 \mid x)\, p(y_2 \mid x)\, p(x)}{p(y_1, y_2)} \propto$$

constant does not depend on x

$$p(x \mid y_1, y_2) \propto p(y_1 \mid x)\, p(y_2 \mid x)\, p(x)$$

$\mathcal{N}_1 \quad \mathcal{N}_2 \quad \mathcal{N}_x$

Homework 2 problem 2

$$\Sigma_f = \left(\Sigma_0^{-1} + \Sigma_1^{-1} + \Sigma_2^{-1}\right)^{-1}$$

$$\mu = \Sigma_f \left(\frac{\mu_0}{\Sigma_0} + \frac{\mu_1}{\Sigma_1} + \frac{\mu_2}{\Sigma_2}\right)$$

Σ_f should be smaller for components greater than 1.

e.g. if all $\Sigma_{0,1,2} = 2$ (greater than 1)
and $\Sigma_{0,1,2} = \frac{1}{2}$ (smaller than 1)

$$\Sigma = \left(\frac{1}{2} + \frac{1}{2} + \frac{1}{2}\right)^{-1} = \frac{2}{3} < 2$$

$$\Sigma = \left(\frac{1}{0.5} + \frac{1}{0.5} + \frac{1}{0.5}\right)^{-1} = \frac{1}{6} < \frac{1}{2}$$

$$\mu = \frac{\Sigma_f}{\Sigma_0}\mu_0 + \frac{\Sigma_f}{\Sigma_1}\mu_1 + \frac{\Sigma_f}{\Sigma_2}\mu_2$$

< 1 < 1 < 1

mean will be smaller too

c)

$$\mathbb{E}[W] = \mathbb{E}[C^\top X] + \mathbb{E}[V] = C^\top \mathbb{E}[X]$$

($\mathbb{E}[V] \to 0$)

$$\mathbb{E}[W] = C^\top \mu_0$$

$$\mathrm{cov}(W) = \mathrm{cov}(C^\top X + V) = \mathrm{cov}(C^\top X) + \mathrm{cov}(V)$$

$$\mathrm{cov}(W) = C^\top \Sigma_0 + R$$

(c) **[5 Points]** We would like to know the probability that the measurement W lies within ε of the expected value of that measurement. Find

$$\mathbb{P}(|W - \mathbb{E}[W]| \leq \varepsilon)\,.$$

Your answer should be in terms of the standard normal cumulative distribution function (CDF), defined for a scalar standard normal random variable Z as $\Phi(a) = Pr(Z \leq a) = \int_{-\infty}^{a} \mathcal{N}(0,1)\,dz$.
Hint: $\mathbb{P}(a \leq Z \leq b) = \Phi(b) - \Phi(a)$ for $Z \sim \mathcal{N}(0,1)$.

W scaled from $\mathcal{N}(0,I)$ as it is $\mathcal{N}(\mu_0, C^T \Sigma_0 + R)$

$$W = \Sigma^{1/2} X_s + \mu$$

$$W - E[W] = \Sigma^{1/2} X_s$$

$$P(\varepsilon \leq W - E[W] \leq \varepsilon) = \Phi(\varepsilon) - \Phi(-\varepsilon)$$

$$P(|W - E[W]| \leq \varepsilon) = (C^T \Sigma_0 + R)^{1/2} \left[\Phi(\varepsilon) - \Phi(-\varepsilon)\right]$$

(d) **[5 Points]** Now suppose we swap the sensor and the new measurement model for W is:

$$W = \cos(X).$$

You are thinking about applying a Kalman filter to the task of estimating the mean and covariance for W over time. Will the KF work? Why or why not?

NO.

W is not a gaussian white noise process, and does not have independent measurements in time. This is a core assumption of the linear Gaussian state space model that the KF is based on.

Q4 (15 points)

Kalman Filter with Feedback Controller

Previously, we assumed a vehicle's control input u_t is deterministic, however in many systems control is state-dependent. Assume U_t is a random variable that is determined through a proportional state feedback policy as

$$U_t = K_p X_t$$

where K_p is the deterministic proportional gain matrix.

Assume the following dynamics and measurement model:

$$X_{t+1} = AX_t + BU_t + W_t$$

$$Y_t = CX_t + V_t$$

where W_t and V_t are zero mean GWN with covariances Q and R, respectively. The noise processes are independent of each other and the initial state X_0.

(a) **[2 Points]** Will the standard Kalman Filter applied to the system matrices A, B, C work for this system? Why or why not?

(b) **[3 Points]** Derive new system matrices A_{cl}, B_{cl}, and C_{cl} to yield a new linear-Gaussian state space model with U_t eliminated (i.e., find the closed loop system model). Can we use the Kalman filter for this model? Why or why not?

(c) **[5 Points]** Suppose you apply the Kalman filter to find $\mu_{t|t}$, $\Sigma_{t|t}$. Using these values, find expressions for the expected value and covariance of the posterior of U_t, given measurement history $Y_{1:t}$.

(d) **[5 Points]** Now suppose your control policy is stochastic, meaning the control input is given by:

$$U_t = K_p X_t + N_t$$

with noise $N_t \sim GWN(0, \Sigma_n)$, independent of X_0 and the W_t and V_t noise processes. Show that you can form a modified linear-Gaussian state space model with A_{cl}, B_{cl}, C_{cl} as above, but with a modified Q_{cl}, for a new process noise covariance. Find Q_{cl}. Assuming you run the KF with this new noise covariance to get $\mu_{t|t}$ and $\Sigma_{t|t}$ find new expressions for $E[U_t \mid Y_{1:t}]$ and $Cov(U_t \mid Y_{1:t})$ for this case.

Q4 (15 points)

Kalman Filter with Feedback Controller

Previously, we assumed a vehicle's control input u_t is deterministic, however in many systems control is state-dependent. Assume U_t is a random variable that is determined through a proportional state feedback policy as

$$U_t = K_p X_t$$

where K_p is the deterministic proportional gain matrix.

Assume the following dynamics and measurement model:

$$X_{t+1} = AX_t + BU_t + W_t$$

$$Y_t = CX_t + V_t$$

where W_t and V_t are zero mean GWN with covariances Q and R, respectively. The noise processes are independent of each other and the initial state X_0.

(a) **[2 Points]** Will the standard Kalman Filter applied to the system matrices A, B, C work for this system? Why or why not?

(b) **[3 Points]** Derive new system matrices A_{cl}, B_{cl}, and C_{cl} to yield a new linear-Gaussian state space model with U_t eliminated (i.e., find the closed loop system model). Can we use the Kalman filter for this model? Why or why not?

(c) **[5 Points]** Suppose you apply the Kalman filter to find $\mu_{t|t}$, $\Sigma_{t|t}$. Using these values, find expressions for the expected value and covariance of the posterior of U_t, given measurement history $Y_{1:t}$.

(d) **[5 Points]** Now suppose your control policy is stochastic, meaning the control input is given by:

$$U_t = K_p X_t + N_t$$

with noise $N_t \sim GWN(0, \Sigma_n)$, independent of X_0 and the W_t and V_t noise processes. Show that you can form a modified linear-Gaussian state space model with A_{cl}, B_{cl}, C_{cl} as above, but with a modified Q_{cl}, for a new process noise covariance. Find Q_{cl}. Assuming you run the KF with this new noise covariance to get $\mu_{t|t}$ and $\Sigma_{t|t}$ find new expressions for $E[U_t \mid Y_{1:t}]$ and $Cov(U_t \mid Y_{1:t})$ for this case.

a) No. B is not affecting the input u_t, but instead acting on the state

b) $A_{cl} = A + BK_p$ $\quad B_{cl} = 0$ $\quad C_{cl} = C$

Yes. Core independence assumptions still hold, and dynamics are linear.

c) $E[U_t \mid Y_{1:t}] = E[K_p X_t \mid Y_{1:t}] = K_p E[X_t \mid Y_{1:t}] = K_p \mu_{t|t}$

$E[U_t \mid Y_{1:t}] = K_p \mu_{t|t}$

$Cov(U_t \mid Y_{1:t}) = Cov(K_p X_t \mid Y_{1:t}) = K_p Cov(X_t \mid Y_{1:t})$ $\quad Cov(U_t \mid Y_{1:t}) = K_p \Sigma_{t|t}$

d) $X_{t+1} = AX_t + BU_t + W_t = AX_t + B(K_p X_t + N_t) + W_t$

$= (A + B_{t+1}) X_t + (BN_t + W_t)$

$Q_{cl} = BN_t + W_t$

$\mu_{t|t-1} = (A_{t-1} + B_{t-1} K_p) \mu_{t-1|t-1}$

$\Sigma_{t|t-1} = (A_{t-1} + B_{t-1} K_p) \Sigma_{t-1|t-1} (A_{t-1} + B_{t-1} K_p)^T + BN_{t-1} + W_{t-1}$

$\mu_{t|t} = \mu_{t|t-1} + k_t (y_t - C_t \mu_{t|t-1})$

$\Sigma_{t|t} = \Sigma_{t|t-1} - k_t C_t \Sigma_{t|t-1}$

$k_t = \Sigma_{t|t-1} C_t^T (C_t \Sigma_{t|t-1} C_t^T + R_t)^{-1}$

Save as before

Q5 (25 points + 5 points extra credit)

Save Mark Watney!

You are an engineer at NASA working on the rescue of Mark Watney. Watney is planning to use his spacesuit to propel himself towards commander Lewis, who will catch him. Because of this risky maneuver, you are tasked with estimating Watney's position with respect to Commander Lewis.

The simplified dynamics are given by:

$$x_{t+1} = x_t + \Delta t\, \dot{x}_t$$
$$\dot{x}_{t+1} = \dot{x}_t + \Delta t\, \ddot{x}_t + W_t$$

where $x_t \in \mathbb{R}^3$, $\dot{x}_t \in \mathbb{R}^3$, $\ddot{x}_t \in \mathbb{R}^3$, are Watney's position, velocity and acceleration, respectively [1]. Watney is also wearing an MPS (Martian Positioning System) receiver, which broadcasts his position with respect to Commander Lewis, via the measurement model:

$$y_t = x_t + V_t$$

With noises $W_t \sim \mathcal{N}(0,\, Q)$ and $V_t \sim \mathcal{N}(0,\, R)$, which are uncorrelated with each other and the initial state.



Figure 1: Rescue of Mark Watney, *from The Martian, 2015*

(a) **[3 Points]** What is the state vector and the system matrices A, B, C?

(b) **[2 Points]** Watney will start at Position $x_0 = [406,\, -203,\, -4.06]$ m, and Velocity $\dot{x}_0 = [0,\, 0,\, 0]$ m/s, and will fly with a constant acceleration $u_t = [-1.0,\ 0.5,\ 0.01]$ m/s^2. Simulate and attach a plot of Watney's trajectory, along with measurements from MPS, to ensure he will rendez-vous within 1m of Commander Lewis at $x_L = [0,\, 0,\, 0]$ after 30 seconds of flight. Assume $\Delta t = 1$ sec, and there is no noise, meaning $Q = 0$, $R = 0$.

Hint 1: You are encouraged to use your code from PS3.

Hint 2: If you are not converging to $x_f = [0, 0, 0]$ within 1 meter, ensure you are using $\Delta t = 1$ sec.

[1] If you were solving this problem in real life, you would use the Hill-Clohessy-Wiltshire Equations, which are used to estimate the relative position and velocity between two objects in a circular orbit

(c) **[2 Points]** You speak up and suggest that Watney's trajectory is not perfect since there is noise! Through field testing you compute $Q = 0.1 \cdot I_3 \text{ m}^2/\text{s}^2$ and $R = 100 \cdot I_3 \text{ m}^2$. Now you simulate Watney's trajectory - does he meet Commander Lewis at the end of his trajectory?

Note: For consistency, please use `np.random.seed(273)`.

(d) **[3 Points]** Nervous, your boss tasks you with implementing a Kalman Filter to estimate Watney's position. Again, we think Watney is starting from

$$x_0 = (406,\ -203,\ -4.06,\ 0,\ 0,\ 0)$$

but we are not sure so we have an initial covariance of

$$P_0 = \begin{bmatrix} 10^3 I_3 \text{ m}^2 & 0 \\ 0 & I_3 \text{ m}^2/\text{s}^2 \end{bmatrix}$$

Attach a plot of the noisy ground truth trajectory, the measurements, and your estimated position.

Note: Your noisy dynamics are exclusively in terms of velocity, so you will need to modify your Q matrix.

(e) **[2 Points]** In addition to not rendezvousing with Lewis, the estimated trajectory has poor performance. In order to fix the Kalman Filter performance, your boss suggests multiplying both the Q, R matrices and initial covariance P_0 by 10. Using your simulation explain why this is not helpful for changing the mean state estimate.

(f) **[6 Points]** Surprised by your result, you decide to prove that if Q, R and P_0 are scaled by the same constant λ, it has the same impact on the mean state estimate as not scaling Q, R, P_0 at all. What is the effect on state covariance?

(g) **[2 Points]** In order to have better filter performance, you claim you need better measurements. Your boss says he can move the MPS satellites for better coverage, and can give you measurements with either $R = 1 \cdot I_3$ or $R = 0.1 \cdot I_3$. Which is better? Defend your answer with simulation results, and briefly explain what is happening in the filter.

(h) **[3 Points]** Your boss is satisfied with the filter performance, but unhappy that Mark is not rendezvousing with Commander Lewis. By only changing Q, R, P_0, how can you get Mark's trajectory to end within 1 meter of Commander Lewis' position.

(i) **[2 Points]** In reality the dynamics function is given by the relative 2-body problem:

$$x_{t+1} = x_t + \Delta t\, \dot{x}_t + \frac{1}{2}\Delta t^2\, \ddot{x}_t$$

$$\dot{x}_{t+1} = \dot{x}_t + \Delta t\, \ddot{x}_t$$

$$\ddot{x}_{t+1} = -\mu \left(\frac{r_L + x_t}{\|r_L + x_t\|^3} - \frac{r_L}{\|r_L\|^3} \right)$$

Where μ is the Martian gravitational constant, and r_L is the time varying position vector between the center of Mars and Lewis. Your boss says this should be compatible with your existing Kalman Filter, though you claim it violates a critical assumption. What assumption does this violate?

(j) **Extra Credit 1 [2 Points]** While these dynamics cannot be used with the standard Kalman Filter, they are solvable with a different mtehod. What method is this?

(k) **Extra Credit 2 [3 points]** What is the physical interpretation to your solution of part (h). Do you think this is tractable in this problem? If not, what would you do to get Watney's trajectory to converge with Commander Lewis?

Q5 (25 points + 5 points extra credit)

Save Mark Watney!

You are an engineer at NASA working on the rescue of Mark Watney. Watney is planning to use his spacesuit to propel himself towards commander Lewis, who will catch him. Because of this risky maneuver, you are tasked with estimating Watney's position with respect to Commander Lewis.

The simplified dynamics are given by:

$$x_{t+1} = x_t + \Delta t\, \dot{x}_t$$
$$\dot{x}_{t+1} = \dot{x}_t + \Delta t\, \ddot{x}_t + W_t$$

where $x_t \in \mathbb{R}^3$, $\dot{x}_t \in \mathbb{R}^3$, $\ddot{x}_t \in \mathbb{R}^3$, are Watney's position, velocity and acceleration, respectively[1]. Watney is also wearing an MPS (Martian Positioning System) receiver, which broadcasts his position with respect to Commander Lewis, via the measurement model:

$$y_t = x_t + V_t$$

With noises $W_t \sim \mathcal{N}(0, Q)$ and $V_t \sim \mathcal{N}(0, R)$, which are uncorrelated with each other and the initial state.



Figure 1: Rescue of Mark Watney, *from The Martian, 2015*

(a) **[3 Points]** What is the state vector and the system matrices A, B, C?

$$a)\ X_{t+1} = X_t + \Delta t\, v_t$$

$$v_{t+1} = v_t + \Delta t\, a_t + w_t$$

State vector $X = \begin{bmatrix} x \\ \dot{x} \end{bmatrix}$

$$\begin{bmatrix} x \\ v \end{bmatrix}_{t+1} = \begin{bmatrix} I_3 & \Delta t I_3 \\ 0 & I_3 \end{bmatrix} \begin{bmatrix} x \\ v \end{bmatrix}_t + \begin{bmatrix} 0 \\ \Delta t I_3 \end{bmatrix} a_t + \begin{bmatrix} 0 \\ \Delta t I_3 \end{bmatrix} w_t$$

$$A = \begin{bmatrix} I_3 & \Delta t I_3 \\ 0 & I_3 \end{bmatrix} \qquad B = \begin{bmatrix} 0 \\ \Delta t I_3 \end{bmatrix}$$

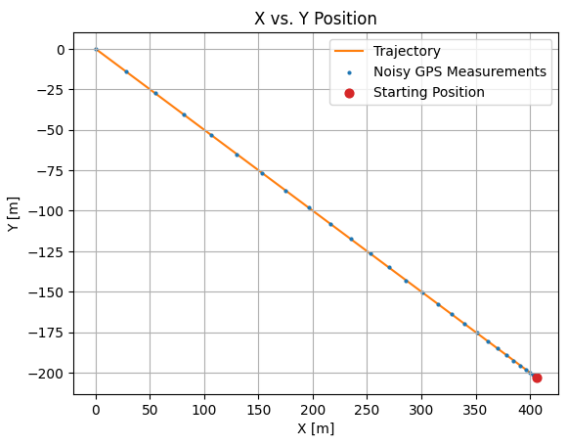
$$y_t = \begin{bmatrix} I & 0 \end{bmatrix} \begin{bmatrix} x \\ v \end{bmatrix}_t + V_t$$

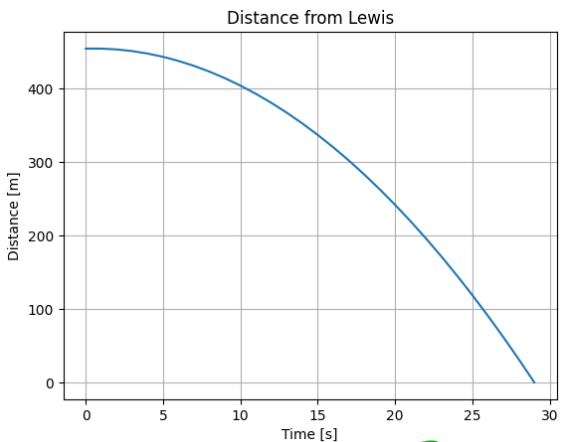
$$C = \begin{bmatrix} I_3 & 0 \end{bmatrix}$$

(b) **[2 Points]** Watney will start at Position $x_0 = [406, -203, -4.06]$ m, and Velocity $\dot{x}_0 = [0, 0, 0]$ m/s, and will fly with a constant acceleration $u_t = [-1.0,\ 0.5,\ 0.01]$ m/s^2. Simulate and attach a plot of Watney's trajectory, along with measurements from MPS, to ensure he will rendez-vous within 1m of Commander Lewis at $x_L = [0, 0, 0]$ after 30 seconds of flight. Assume $\Delta t = 1$ sec, and there is no noise, meaning $Q = 0$, $R = 0$.

Hint 1: You are encouraged to use your code from PS3.

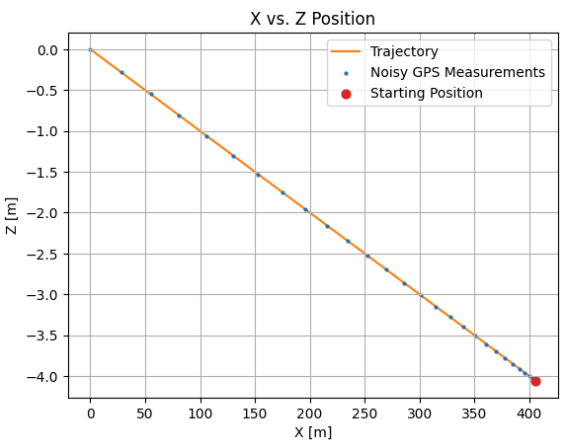
Hint 2: If you are not converging to $x_f = [0, 0, 0]$ within 1 meter, ensure you are using $\Delta t = 1$ sec.





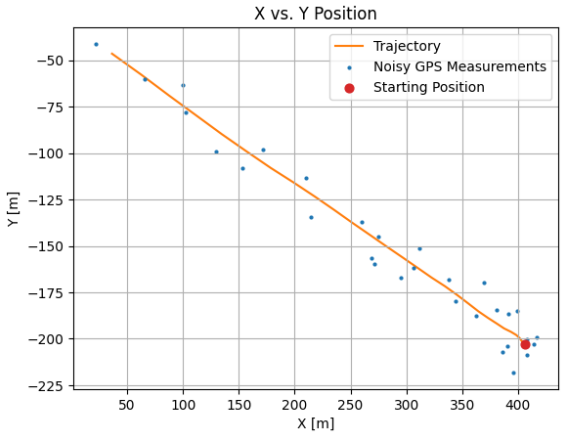
He meets her

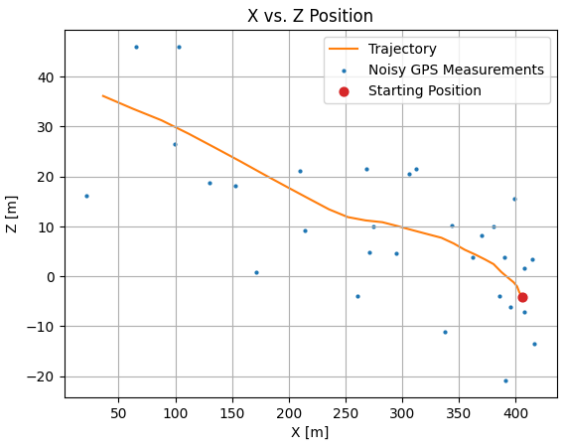


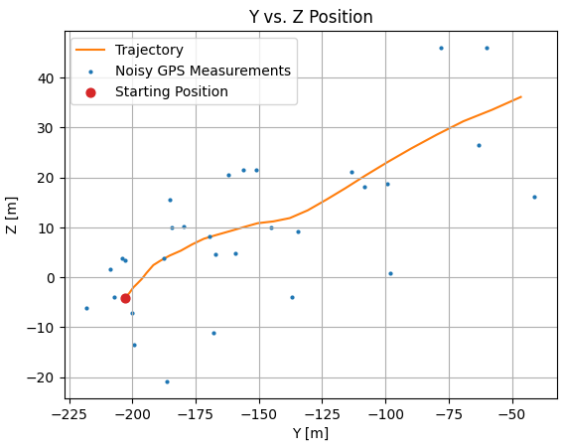


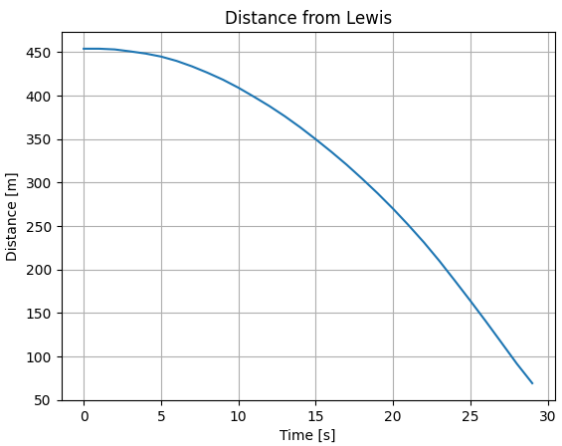
(c) **[2 Points]** You speak up and suggest that Watney's trajectory is not perfect since there is noise! Through field testing you compute $Q = 0.1 \cdot I_3 \text{ m}^2/\text{s}^2$ and $R = 100 \cdot I_3 \text{ m}^2$. Now you simulate Watney's trajectory - does he meet Commander Lewis at the end of his trajectory?

Note: For consistency, please use `np.random.seed(273)`.









He does NOT meet her

(d) **[3 Points]** Nervous, your boss tasks you with implementing a Kalman Filter to estimate Watney's position. Again, we think Watney is starting from

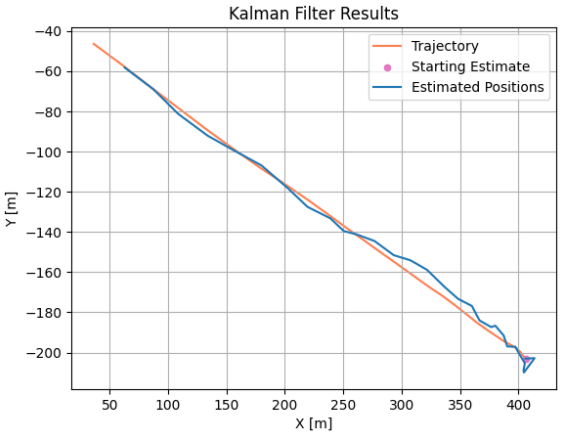
$$x_0 = (406, -203, -4.06, 0, 0, 0)$$

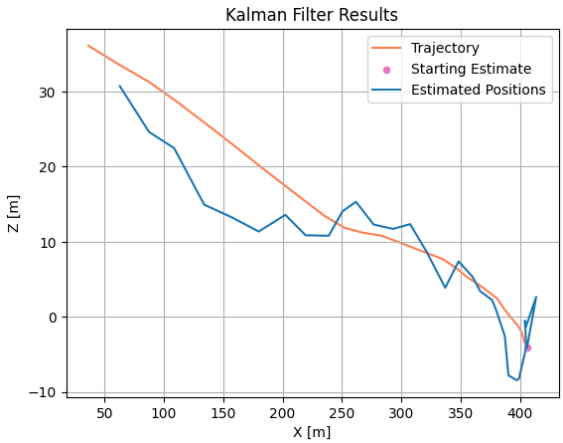
but we are not sure so we have an initial covariance of

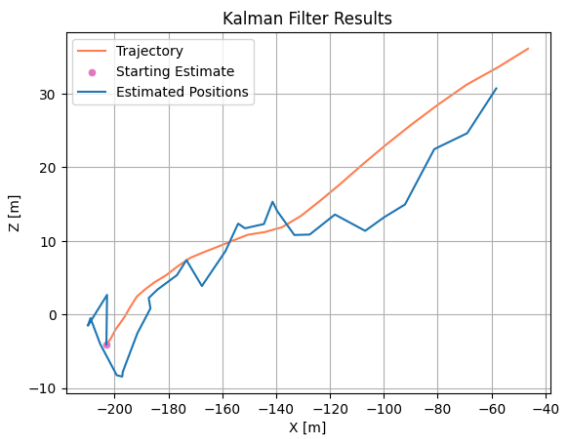
$$P_0 = \begin{bmatrix} 10^3 I_3 \text{ m}^2 & 0 \\ 0 & I_3 \text{ m}^2/\text{s}^2 \end{bmatrix}$$

Attach a plot of the noisy ground truth trajectory, the measurements, and your estimated position.

Note: Your noisy dynamics are exclusively in terms of velocity, so you will need to modify your Q matrix.

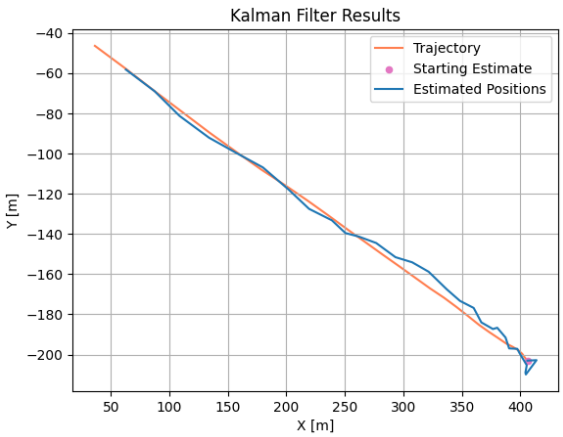


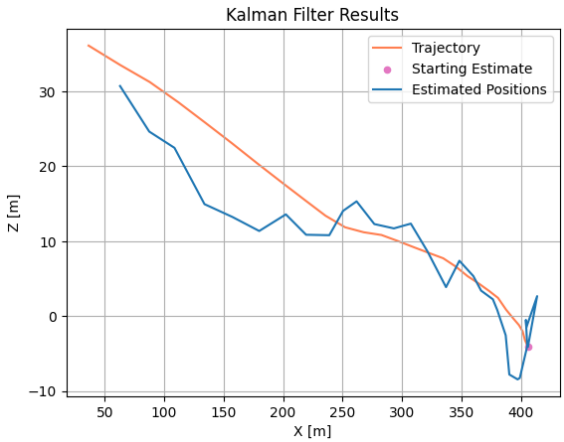


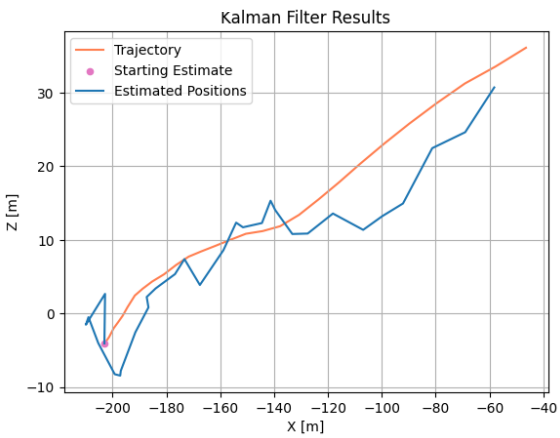


(e) **[2 Points]** In addition to not rendezvousing with Lewis, the estimated trajectory has poor performance. In order to fix the Kalman Filter performance, your boss suggests multiplying both the Q, R matrices and initial covariance P_0 by 10. Using your simulation explain why this is not helpful for changing the mean state estimate.

Same exact plots. Because by scaling all covariances by the same amount, the filter does not change how much it weights/"trusts" the dynamics model vs. the measurements, so it will have no effect unless some change their weights by a different amount.







(f) **[6 Points]** Surprised by your result, you decide to prove that if Q, R and P_0 are scaled by the same constant λ, it has the same impact on the mean state estimate as not scaling Q, R, P_0 at all. What is the effect on state covariance?

original $P_0 =$

```
[[22.26890965  0.          0.          2.78686679  0.          0.        ]
 [ 0.         22.26890965  0.          0.          2.78686679  0.        ]
 [ 0.          0.         22.26890965  0.          0.          2.78686679]
 [ 2.78686679  0.          0.          0.7991375   0.          0.        ]
 [ 0.          2.78686679  0.          0.          0.7991375   0.        ]
 [ 0.          0.          2.78686679  0.          0.          0.7991375 ]]
```

scaled $P_0 =$

```
[[222.6891   0.       0.      27.8687   0.       0.    ]
 [  0.     222.6891   0.       0.      27.8687   0.    ]
 [  0.       0.     222.6891   0.       0.      27.8687]
 [ 27.8687   0.       0.       7.9914   0.       0.    ]
 [  0.      27.8687   0.       0.       7.9914   0.    ]
 [  0.       0.      27.8687   0.       0.       7.9914]]
```

Scaled by 10

(g) **[2 Points]** In order to have better filter performance, you claim you need better measurements. Your boss says he can move the MPS satellites for better coverage, and can give you measurements with either $R = 1 \cdot I_3$ or $R = 0.1 \cdot I_3$. Which is better? Defend your answer with simulation results, and briefly explain what is happening in the filter.

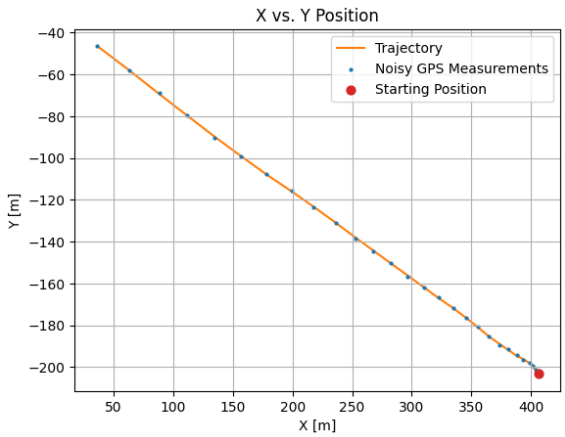
$R = 0.1 \cdot I_3$ is better. Since R is the covariance of the measured noise, having less noise in sensor measurements means the filter will trust the (presumably) more accurate MPS more (over the dynamics).

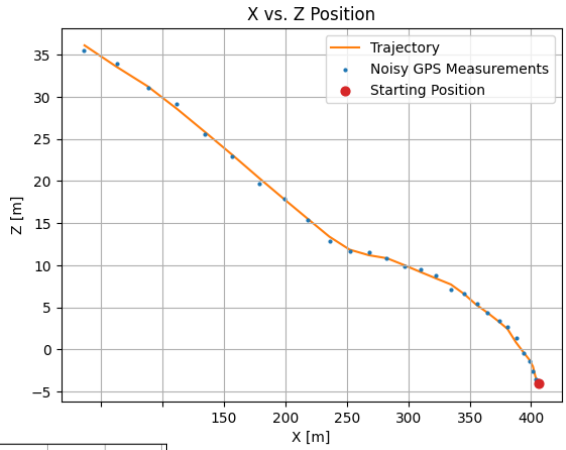
But if the dynamics model is off, then it will just simulate the wrong trajectory.

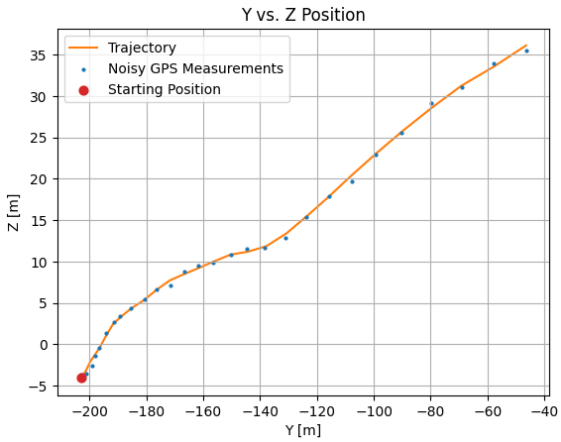
Also a little confused if we use $Q = 0$ or $Q = 0.1\ I_3$. it doesn't make sense to change the kalman filter if the trajectory itself never reaches Lewis. Proceeding with analyzing $Q = 0$ and $Q = 0.1\ I_3$

$Q = 0.1 I_3$

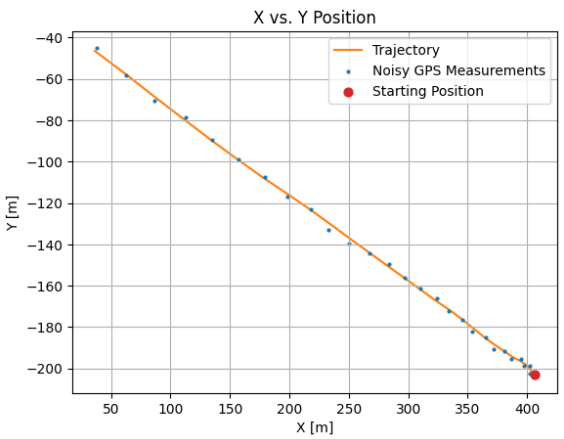
measurements a lot closer to true trajectory with $R = 0.1 I_3$

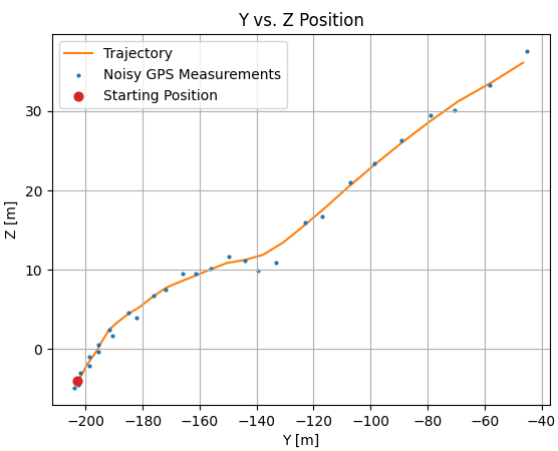




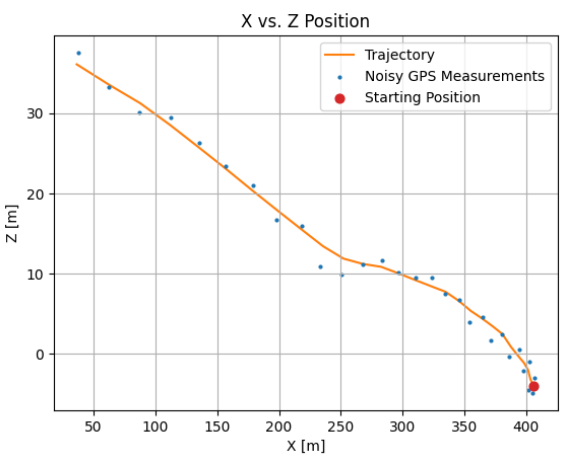


$R = 0.1 I_3$





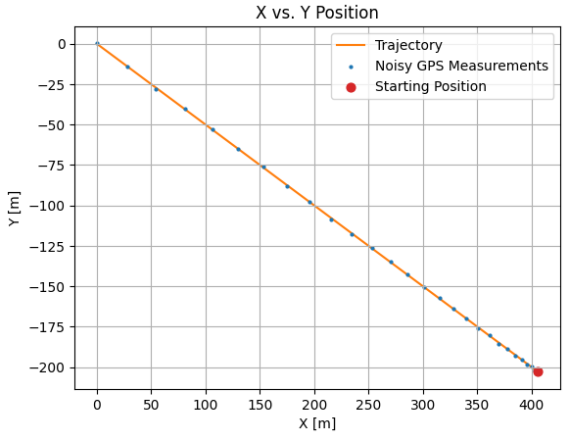
$R = I_3$

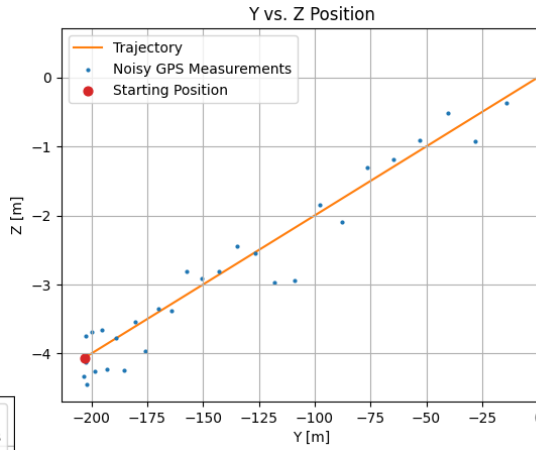


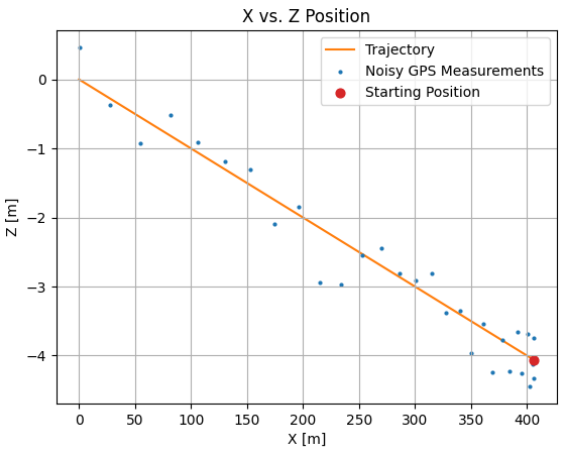
$Q = 0$

measurements a lot closer to true trajectory with $R = 0.1 I_3$

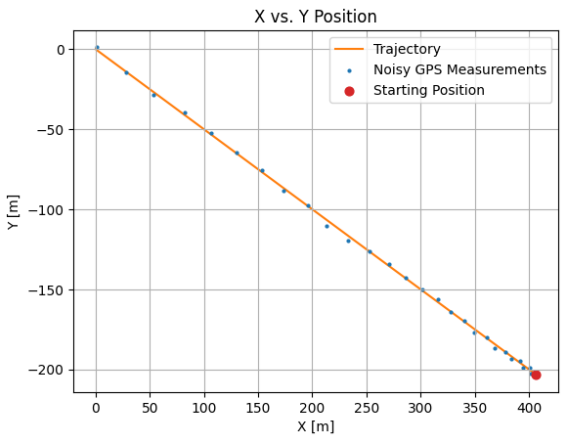
$R = 0.1 I_3$

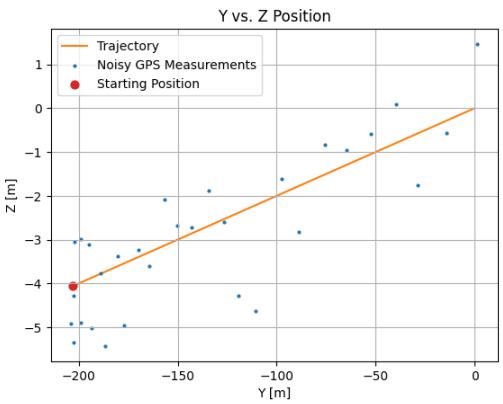


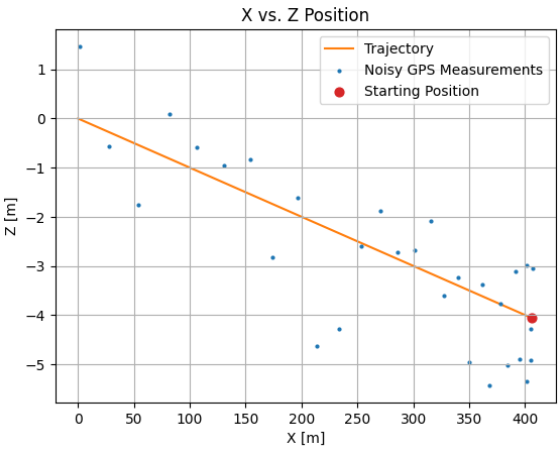




$R = I_3$







(h) **[3 Points]** Your boss is satisfied with the filter performance, but unhappy that Mark is not rendezvousing with Commander Lewis. By only changing Q, R, P_0, how can you get Mark's trajectory to end within 1 meter of Commander Lewis' position.

$Q=0$ By decreasing R, the filter will trust the measurements more, and converge. Increasing Q will also have a similar effect

$Q=0.1I_3$ Unless the trajectory randomly hits Lewis (since it is random) there is no way to reliably make Mark's trajectory hit Lewis.

(i) **[2 Points]** In reality the dynamics function is given by the relative 2-body problem:

$$x_{t+1} = x_t + \Delta t\, \dot{x}_t + \frac{1}{2}\Delta t^2 \ddot{x}_t$$

$$\dot{x}_{t+1} = \dot{x}_t + \Delta t\, \ddot{x}_t$$

$$\ddot{x}_{t+1} = -\mu\left(\frac{r_L + x_t}{\|r_L + x_t\|^3} - \frac{r_L}{\|r_L\|^3}\right)$$

Where μ is the Martian gravitational constant, and r_L is the time varying position vector between the center of Mars and Lewis. Your boss says this should be compatible with your existing Kalman Filter, though you claim it violates a critical assumption. What assumption does this violate?

There are nonlinear equations. A, B, C, D assume linear dynamics and measurement model, but $\ddot{x}$ is not.

(j) **Extra Credit 1 [2 Points]** While these dynamics cannot be used with the standard Kalman Filter, they are solvable with a different mtehod. What method is this?

An Extended Kalman Filter could be used, in which the dynamics and measurement are linearized around the current state, and the A and C matrices become Jacobians of the functions $x_{t+1} = f(x_t, u_t)$ and $y_t = g(x_t, u_t)$ respectively

(k) **Extra Credit 2 [3 points]** What is the physical interpretation to your solution of part (h). Do you think this is tractable in this problem? If not, what would you do to get Watney's trajectory to converge with Commander Lewis?

this makes sense for the $Q=0.1I_3$ case. In this case, the simulated trajectory is too noisy to physically make it to Lewis, so by simulating with a smaller Q, the trajectory can make it (eg. in the $Q=0$ case).

Controlled Bayes Filter

A discrete state $X_t \in \{1, 2, 3\}$ evolves according to a controlled transition model that depends on a control input $u_t \in \{\text{N}, \text{F}\}$ (No-move vs Forward). The "forward" direction is $1 \to 2 \to 3 \to 1$.

Controlled transition model.

- If $u_t = \text{N}$: $p(x_t = x_{t-1}) = 0.9$, and the remaining 0.1 probability is split equally among the other two states.
- If $u_t = \text{F}$: with probability 0.8 you go forward (i.e., $x_t = x_{t-1} + 1 \bmod 3$), and with probability 0.2 you stay (i.e., $x_t = x_{t-1}$).

Observation model. The observation is $Y_t \in \{1, 2, 3\}$ with

$$p(Y_t = y \mid X_t = x) = \begin{cases} 0.7, & y = x, \\ 0.15, & y \neq x \text{ (each of the other two values).} \end{cases}$$

Initial belief:

$$b_0 = [0.2,\ 0.5,\ 0.3]$$

corresponding to states $[1, 2, 3]$.

Controls and observations:

$$u_1 = \text{F},\ y_1 = 2; \qquad u_2 = \text{N},\ y_2 = 2.$$

(a) **[6 Points]** Write the Bayes filter recursion for

$$b_t(x) = p(X_t = x \mid y_{1:t}, u_{1:t}),$$

including a prediction step using $p(x_t \mid x_{t-1}, u_t)$ and an update step using $p(y_t \mid x_t)$.

(b) **[10 Points]** Compute b_1 and b_2 numerically.

(c) **[4 Points]** Let $u_3 = \text{F}$. Compute the predicted belief

$$\bar{b}_3(x) = p(X_3 = x \mid y_{1:2}, u_{1:3})$$

(i.e., prediction only; no measurement update at time 3).

$$p(x_t \mid x_{t-1}, u_t) \qquad p(y_t \mid x_t) \qquad p(x|y) = \frac{p(y|x)\,p(x)}{p(y)}$$

$$p(x_t \mid y_{1:t-1}, u_t) = \int_{x_{t-1}} p(x_t \mid x_{t-1}, u_t)\, p(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}$$

$$p(x_t \mid y_{1:t}) = \frac{p(y_t \mid x_t)\, p(x_t \mid y_{1:t-1})}{\int_{x_t} p(y_t, x_t)\, p(x_t \mid y_{1:t-1})\, dx_t}$$

b) $p(x_1 \mid y_0, u_1) = p(x_1 \mid x_0 = 1, \text{F})\, p(x_0 = 1 \mid y_0)$ (0.2) belief gives at 1

$+\ p(x_1 \mid x_0 = 2, \text{F})\, p(x_0 = 2 \mid y_0)$ (0.5)

$+\ p(x_1 \mid x_0 = 3, \text{F})\, p(x_0 = 3 \mid y_0)$ (0.5)

$p(x_1 = 1 \mid y_0, u_1) = (0.2)(0.2) + (0)(0.5) + (0.8)(0.2) = 0.28$

$p(x_1 = 2 \mid y_0, u_1) =$

$p(x_1 = 2 \mid x_0 = 1, \text{F})\, p(x_0 = 1 \mid y_0)$ (0.2) belief gives at 1

$+\ p(x_1 = 2 \mid x_0 = 2, \text{F})\, p(x_0 = 2 \mid y_0)$ (0.5)

$+\ p(x_1 = 2 \mid x_0 = 3, \text{F})\, p(x_0 = 3 \mid y_0)$ (0.5)

$$p(x_1=2 \mid y_0, u_1) = (0.8)(0.2) + (0.2)(0.5) + (0) = 0.26$$

$$p(x_1=3 \mid y_0, u_1) = p(X=3 \mid X=1, F)\, p(x=1 \mid y_0) + p(X=3 \mid X=2, F)\, p(x=2 \mid y_0) + p(X=3 \mid x=3, F)\, p(x=3 \mid y_0)$$

0.2 — belief given at 1; 0.5; 0.5

$$p(x_1=3 \mid y_0, u_1) = (0) + (0.8)(0.5) + (0.2)(0.3) = 0.46$$

$$b_1 = \begin{bmatrix} 0.28 \\ 0.26 \\ 0.46 \end{bmatrix}$$ pre

$$p(x_t \mid y_1) = \frac{p(y_1 \mid x_1)\, p(x_1 \mid y_0)}{p(y_1 \mid x=1)\, p(x=1) + p(y_1 \mid x=2)\, p(x=2) + p(y_1 \mid x=3)\, p(x=3)}$$

(2) (2) (2) (1) (2)

$$= \frac{\begin{bmatrix} 0.15 \cdot 0.28 \\ 0.7 \cdot 0.26 \\ 0.15 \cdot 0.46 \end{bmatrix}}{(0.15)(0.28) + (0.7)(0.26) + (0.15)(0.46)}$$

$$b_1 = \begin{bmatrix} 0.1437 \\ 0.6212 \\ 0.2355 \end{bmatrix}$$

repeating

$$P(X_2=1 \mid y_1, y_2) = P(X_2=1 \mid X=1, N)\, P(X=1 \mid y_1) + P(X=1 \mid X=2, N)\, P(X=2 \mid y_1) + P(X=1 \mid X=3, N)\, P(X=3 \mid y_1)$$

$$P(x_2=1 \mid y_0, y_1) = (0.9)(0.1437) + (0.1)(0.6212) + (0.1)(0.2355) = 0.2147$$

$$P(X_2=2 \mid y_1, y_2) = P(X_2=2 \mid X=1, N)\, P(X=1 \mid y_1) + P(X=2 \mid X=2, N)\, P(X=2 \mid y_1) + P(X=2 \mid X=3, N)\, P(X=3 \mid y_1)$$

$$P(x_1=2 \mid y_0, y_1) = (0.1)(0.1437) + (0.9)(0.6212) + (0.1)(0.2355) = 0.5969$$

$$P(x_1=3 \mid y_0, y_1) = (0.1)(0.1437) + (0.1)(0.6212) + (0.9)(0.2355) = 0.2884$$

$$b_{2,prd} = \begin{bmatrix} 0.2147 \\ 0.5969 \\ 0.2884 \end{bmatrix}$$

$$P(x_t \mid y_1) = \frac{P(y_2 \mid x_2)\, P(x_2 \mid y_1)}{P(y_2 \mid x=1)\, P(x=1) + P(y_2 \mid x=2)\, P(x=2) + P(y_2 \mid x=3)\, P(x=3)}$$

$$= \frac{\begin{bmatrix} 0.15 & 0.2147 \\ 0.7 & 0.5969 \\ 0.15 & 0.2884 \end{bmatrix}}{(0.15)(0.1437) + (0.7)(0.6212) + (0.15)(0.2355)}$$

$$b_2 = \begin{bmatrix} 0.0653 \\ 0.8470 \\ 0.0877 \end{bmatrix}$$

Same process as first

$$P(x_3=1 \mid y_2, y_3) = (0.2)(b_2(1)) + (0)(b_2(2)) + (0.8)(b_2(3)) = 0.0988$$

$$P(x_3=2 \mid y_2, y_3) = (0.8)(b_2(1)) + (0.2)(b_2(2)) + (0) = 0.2841$$

$$P(x_3=3 \mid y_2, y_3) = 0 + 0.8(b_2(2)) + 0.2(b_2(3)) = 0.6952$$

$$b_3 = \begin{bmatrix} 0.0988 \\ 0.2841 \\ 0.6952 \end{bmatrix}$$

```
clear all
clc
close all

T_N = [0.9 0.05 0.05; 0.05 0.9 0.05; 0.05 0.05 0.9];
T_F = [0.2 0.8 0; 0 0.2 0.8; 0.8 0 0.2];

y2_x = transpose([0.15 0.7 0.15]);
b0 = transpose([0.2 0.5 0.3]);
b1_pre = T_F * b0
b1 = y2_x .* b1_pre / sum(y2_x .* b1_pre)

b2_pre = T_N * b1
b2 = y2_x .* b2_pre / sum(y2_x .* b2_pre)

b3_pre = T_F * b2
```

```
b1_pre =

    0.4400
    0.3400
    0.2200


b1 =

    0.1958
    0.7062
    0.0979


b2_pre =

    0.2165
    0.6503
    0.1332


b2 =

    0.0640
    0.8967
    0.0394


b3_pre =

    0.7301
    0.2108
    0.0590
```

Published with MATLAB® R2025b

In [117…
```
import numpy as np
from numpy.random import randn
from scipy.linalg import sqrtm, inv
import matplotlib.pyplot as plt
np.random.seed(273)
```

In [117…
```
def epsilon_and_nu(P: float, Sigma: np.ndarray):
    nu = 1/ (2 * np.pi * np.sqrt(np.linalg.det(Sigma)))
    epsilon = (1-P) * nu

    return epsilon, nu

def in_error_ellipse(x: np.ndarray, mu: np.ndarray, Sigma: np.ndarray, P=0.9
    epsilon, nu = epsilon_and_nu(P, Sigma)
    rhs = -2*np.log(epsilon / nu)   # = -2 log(1-P)
    lhs = (x - mu).T @ np.linalg.inv(Sigma) @ (x - mu)
    return lhs <= rhs

def plot_gaussian(mu, Sigma, N, show=True):
    x_arr = randn(N,2) @ sqrtm(Sigma) + mu

    x_in = np.array([x for x in x_arr if in_error_ellipse(x, mu, Sigma)])
    x_out = np.array([x for x in x_arr if not in_error_ellipse(x, mu, Sigma)

    e, _ = epsilon_and_nu(0.95, Sigma)
    print(f"Percentage of points in ellipse {len(x_in) / N * 100}")
    plt.scatter(x_in[:,0], x_in[:,1], marker='.')
    plt.scatter(x_out[:,0], x_out[:,1], marker='.')
    plt.title("95% Confidence Error Ellipse")
    if show:
        plt.show()

def get_ellipse_points(mu: np.ndarray, Sigma: np.ndarray, P=0.95, N=200):
    t = np.linspace(0, 2*np.pi, N)
    circle = np.vstack((np.cos(t), np.sin(t)))
    rhs = -2*np.log(1-P)
    ellipse = np.sqrt(rhs) * circle
    scaled_ellipse = sqrtm(Sigma) @ ellipse # circle -> ellipse
    return scaled_ellipse + mu.reshape(2,1)
```

Problem 1

Finding mu_x_y

In [118…
```
mu_x = np.array([0, 0]).reshape((2,1))
mu_y = np.array([0, 0]).reshape((2,1))

Sigma_xx = np.array([
    [0.7, 0.73],
    [0.73, 1.1]])
Sigma_yy = np.array([
```

```
    [0.7, 0.19],
    [0.19, 0.16]])
Sigma_xy = np.array([
    [0.63, 0.23],
    [0.72, 0.31]])
Sigma_yy_inv = inv(Sigma_yy)
Sigma_yx = Sigma_xy.T

y = np.array([0.27, 0.62]).reshape((2,1))

print(Sigma_yy_inv)
print(Sigma_yx)
```

```
[[ 2.108  -2.5033]
 [-2.5033  9.2227]]
[[0.63 0.72]
 [0.23 0.31]]
```

In [118…

```
mu_x_y = mu_x + Sigma_xy @ Sigma_yy_inv @ (y - mu_y)
Sigma_x_y = Sigma_xx - Sigma_xy @ Sigma_yy_inv @ Sigma_yx

print(mu_x_y)
print(Sigma_x_y)
```

```
[[0.5405]
 [0.8554]]
[[0.1009 0.0197]
 [0.0197 0.2384]]
```

Plotting

In [118…

```
plt.plot(*get_ellipse_points(mu_x, Sigma_xx), color='tab:blue', label=r"Pric

plt.plot(*get_ellipse_points(mu_x_y, Sigma_x_y), color='tab:orange', label=r
plt.scatter(*mu_x_y, color='tab:orange', label=r"Posterior mean $\mu_{x|y}$"

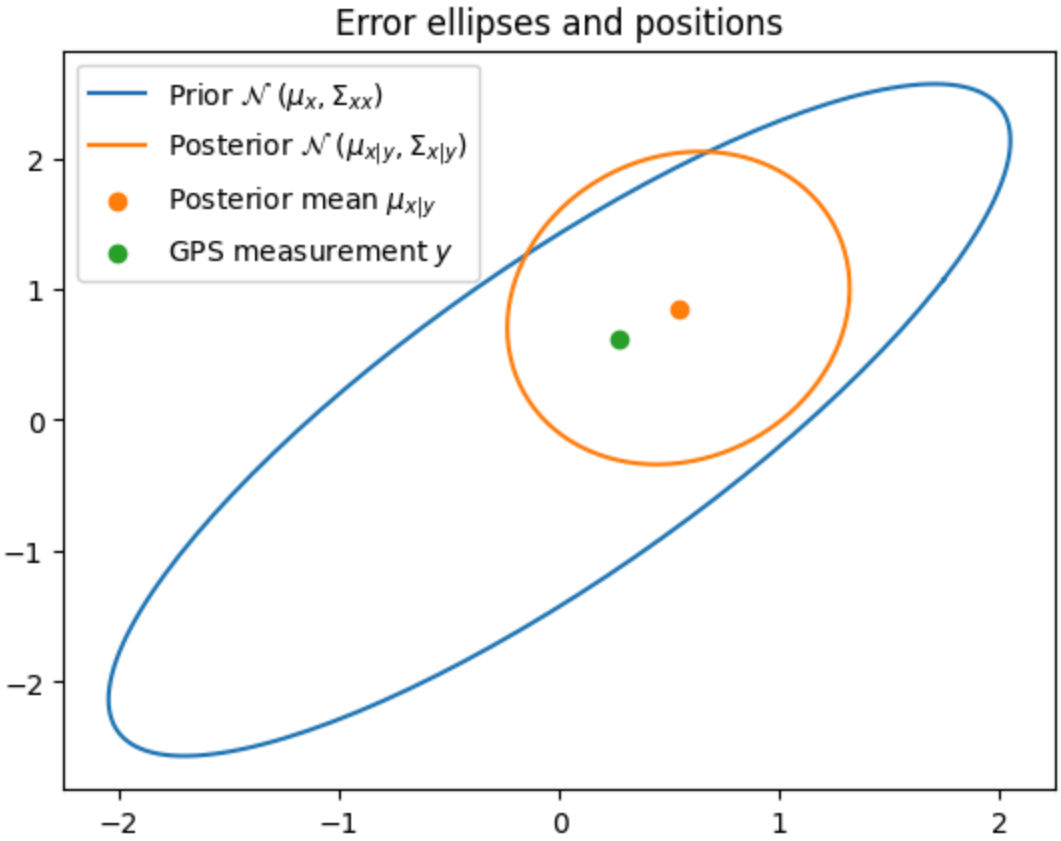
plt.scatter(*y, color='tab:green', label="GPS measurement $y$")

plt.legend()

plt.title("Error ellipses and positions")

plt.show()

# plt.scatter(*mu_x, color='orange')
# plot_gaussian(mu_x_y, Sigma_x_y, 1000, )
```



Problem 3

Creating trajectories

In [118…
```
dt = 1
# Q = np.zeros((4,4))
# Q[2:4, 2:4] = np.eye(2)

# R = 9*np.eye(2)
Q = np.zeros((6,6))
R = np.zeros((3,3))

def u_input(_):
    # return -2.5 * np.array([
    #     np.cos(0.05*t), np.sin(0.05*t)
    # ])
    return np.array([-1, 0.5, 0.01])
```

In [118…
```
A = np.zeros((6,6))
A[0:3, 0:3] = np.eye(3)
A[0:3, 3:6] = dt*np.eye(3)
A[3:6, 3:6] = np.eye(3)

B = np.zeros((6,3))
B[3:6, 0:3] = np.eye(3) * dt

C = np.zeros((3,6))
```

```
C[0:3, 0:3] = np.eye(3)

x0 = np.array([406, -203, -4.06, 0, 0, 0])
```

In [118…
```
C
```

Out[118…
```
array([[1., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0.]])
```

In [118…
```
print(C @ x0)
```

```
[ 406.   -203.     -4.06]
```

In [121…
```
def noise_W(Q, seed=None):
    if seed is not None:
        np.random.seed(seed)

    return np.random.multivariate_normal(
        mean=np.zeros(Q.shape[0]),
        cov=Q
    )

def noise_V(R, seed=None):
    if seed is not None:
        np.random.seed(seed)

    return np.random.multivariate_normal(
        mean=np.zeros(R.shape[0]),
        cov=R
    )

print(noise_W(Q))
print(noise_V(R))
```

```
[ 0.      0.      0.      0.0824  0.314  -0.2267]
[ 1.8153  4.3816 -4.1649]
```

In [118…
```
N = 30 # Timesteps of 1 second

class Trajectory:
    def __init__(self, x, u, meas):
        self.x_arr = x
        self.u_arr = u
        self.meas_arr = meas

def one_trajectory(N, Q=Q, R=R, show=True, return_results = True):

    meas_arr = np.zeros((N,3))
    u_arr = np.zeros((N,3))
    x_arr = np.zeros((N,6))


    x_arr[0] = x0
    print(x0)
    meas_arr[0] = C @ x0 + noise_V(R)
```

```
    for t in range(N-1):
        u_arr[t] = u_input(t).T

        x_arr[t+1] = A @ x_arr[t] + B @ u_arr[t] + noise_W(Q)
        # print(x_arr[t+1] - x_arr[t])
        meas_arr[t+1] = C @ x_arr[t+1] + noise_V(R)



    if show:
        dimA = 0
        dimB = 1
        plt.plot(x_arr[:,dimA], x_arr[:,dimB], label="Trajectory", color="ta
        plt.scatter(meas_arr[:,dimA], meas_arr[:,dimB], label="Noisy GPS Mea
        plt.scatter([x0[dimA]], [x0[dimB]], color="tab:red", label="Starting
        plt.xlabel("X [m]")
        plt.ylabel("Y [m]")
        plt.title("X vs. Y Position")
        plt.legend(("Trajectory", "Noisy GPS Measurements", "Starting Positi
        plt.grid()
        plt.show()

        dimA = 1
        dimB = 2
        plt.plot(x_arr[:,dimA], x_arr[:,dimB], label="Trajectory", color="ta
        plt.scatter(meas_arr[:,dimA], meas_arr[:,dimB], label="Noisy GPS Mea
        plt.scatter([x0[dimA]], [x0[dimB]], color="tab:red", label="Starting
        plt.xlabel("Y [m]")
        plt.ylabel("Z [m]")
        plt.title("Y vs. Z Position")
        plt.legend(("Trajectory", "Noisy GPS Measurements", "Starting Positi
        plt.grid()
        plt.show()

        dimA = 0
        dimB = 2
        plt.plot(x_arr[:,dimA], x_arr[:,dimB], label="Trajectory", color="ta
        plt.scatter(meas_arr[:,dimA], meas_arr[:,dimB], label="Noisy GPS Mea
        plt.scatter([x0[dimA]], [x0[dimB]], color="tab:red", label="Starting
        plt.xlabel("X [m]")
        plt.ylabel("Z [m]")
        plt.title("X vs. Z Position")
        plt.legend(("Trajectory", "Noisy GPS Measurements", "Starting Positi
        plt.grid()
        plt.show()
        # plt.show()

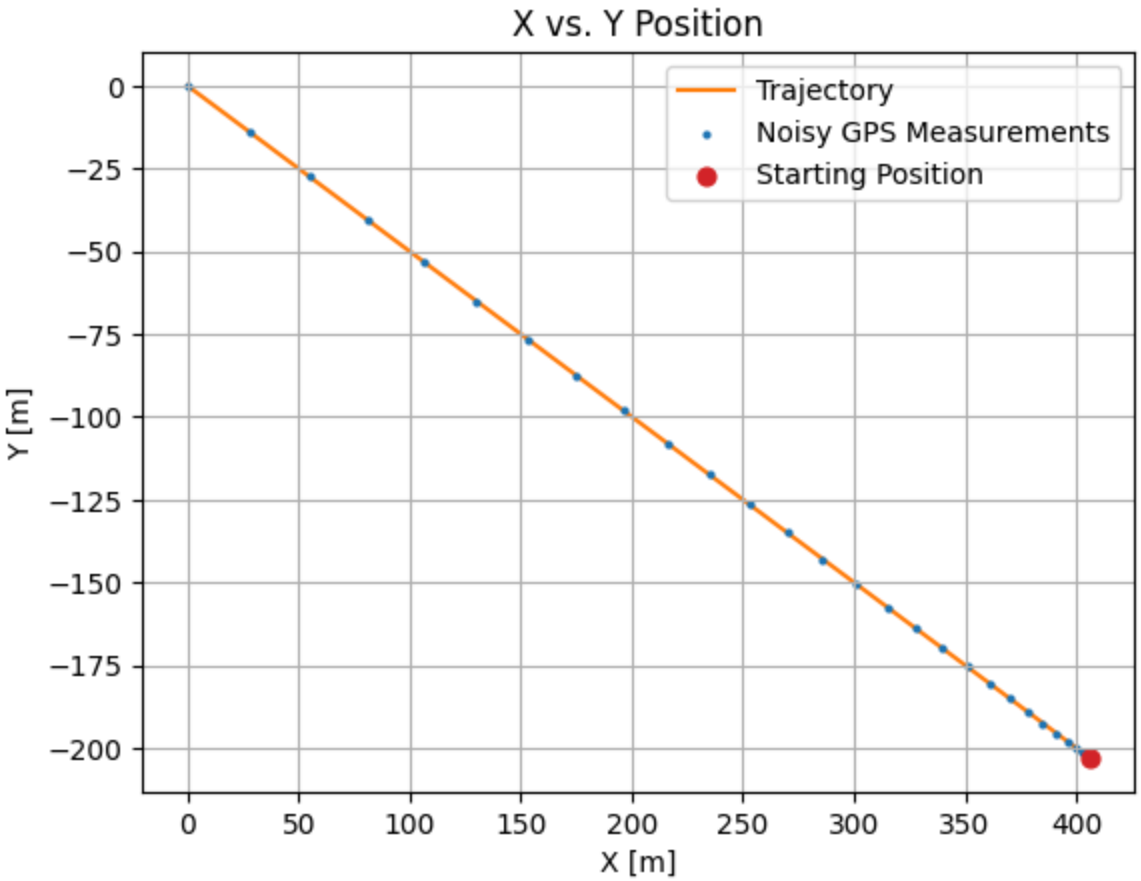
    if return_results:
        return Trajectory(x_arr, u_arr, meas_arr)
```

In [118…

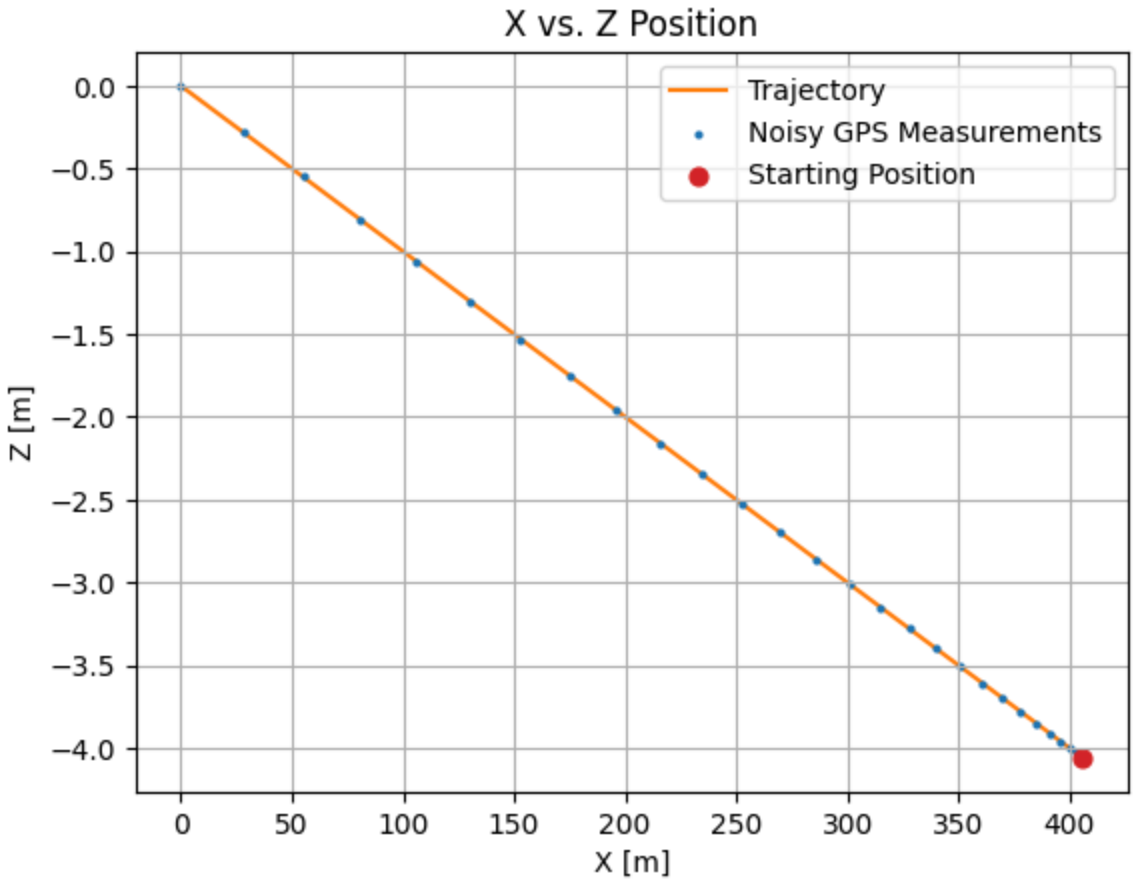
```
data_arrs: list[Trajectory] = []
np.random.seed(273)
traj = one_trajectory(30)
```

```
# data_arrs.append(one_trajectory(100))
# plt.grid()
```

```
[ 406.   -203.     -4.06    0.       0.       0.  ]
```

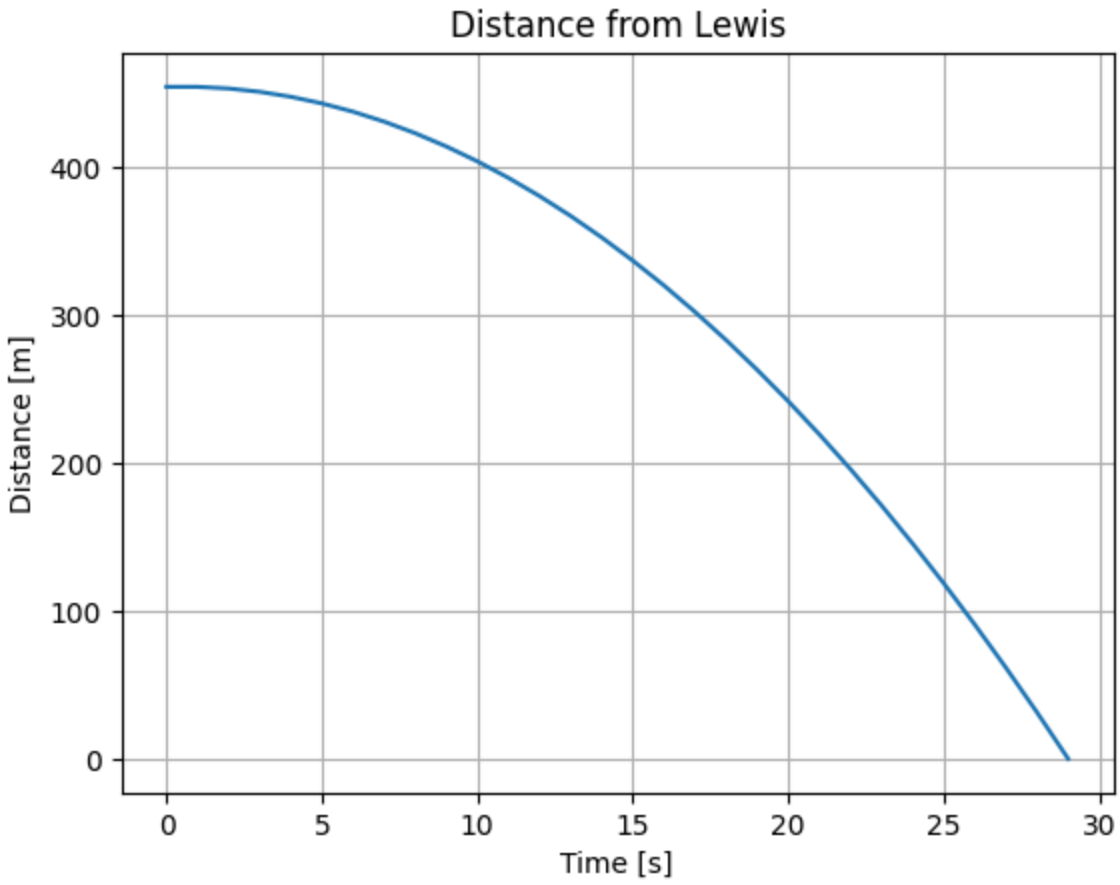



Y vs. Z Position
Trajectory
Noisy GPS Measurements
Starting Position
Z [m]
Y [m]
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
−3.5
−4.0
−200
−175
−150
−125
−100
−75
−50
−25
0


X vs. Z Position
Trajectory
Noisy GPS Measurements
Starting Position
Z [m]
X [m]
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
−3.5
−4.0
0
50
100
150
200
250
300
350
400

In [119…
```
dist = []
for i in traj.x_arr:
    dist.append(np.linalg.norm(i[:3]))
dist = np.asarray(dist)
plt.plot(dist)
plt.grid()
plt.title("Distance from Lewis")
plt.xlabel("Time [s]")
plt.ylabel("Distance [m]")
```

Out[119… `Text(0, 0.5, 'Distance [m]')`



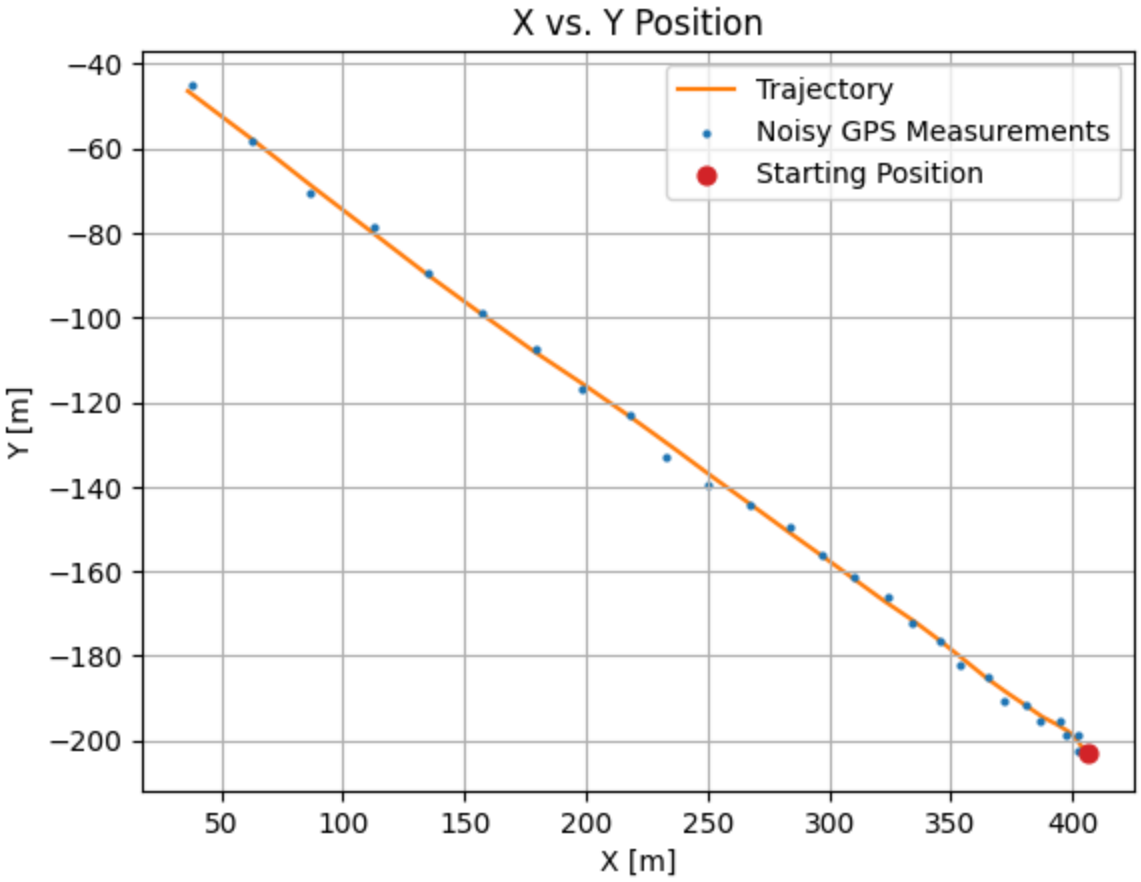
In [122…
```
Q = np.zeros((6,6))
Q[3:6, 3:6] = 0.1*np.eye(3)
Q_OG = Q
# R = 100*np.eye(3)
R = 100*np.eye(3)
# R_OG = R

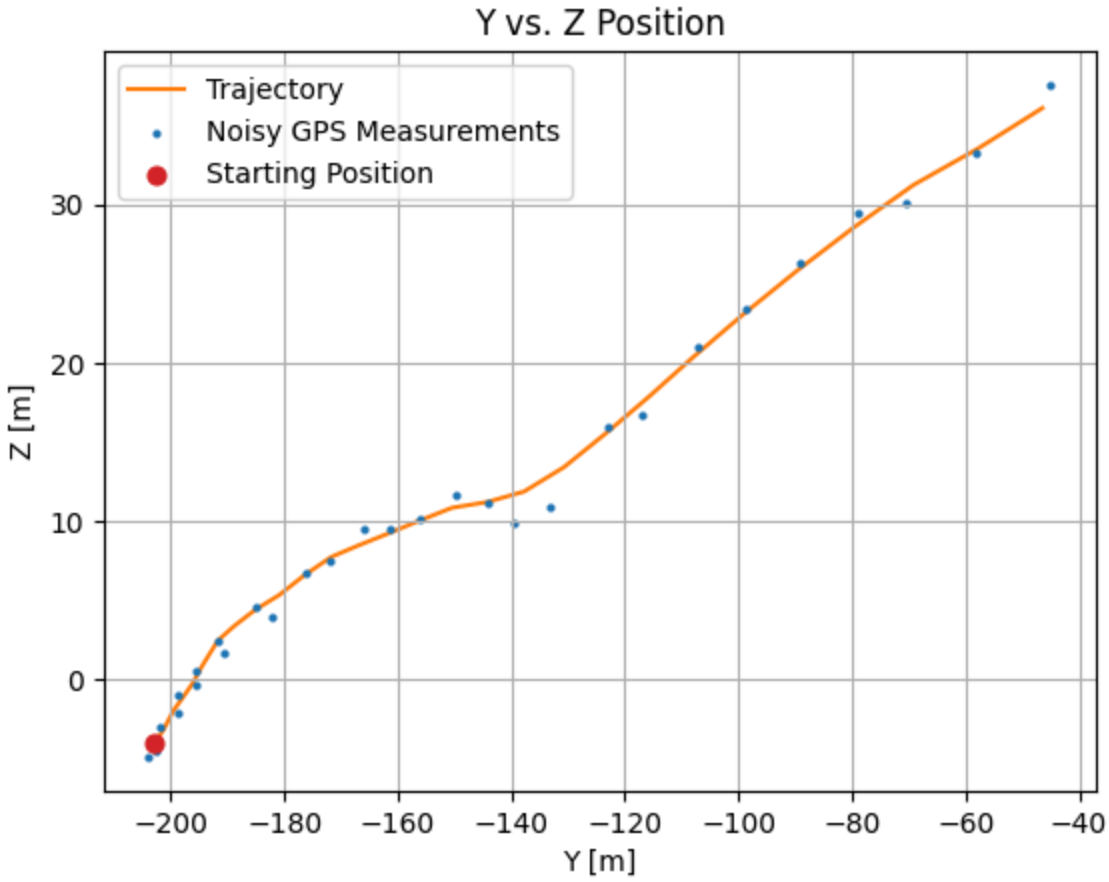
print(Q)
print(R)
np.random.seed(273)

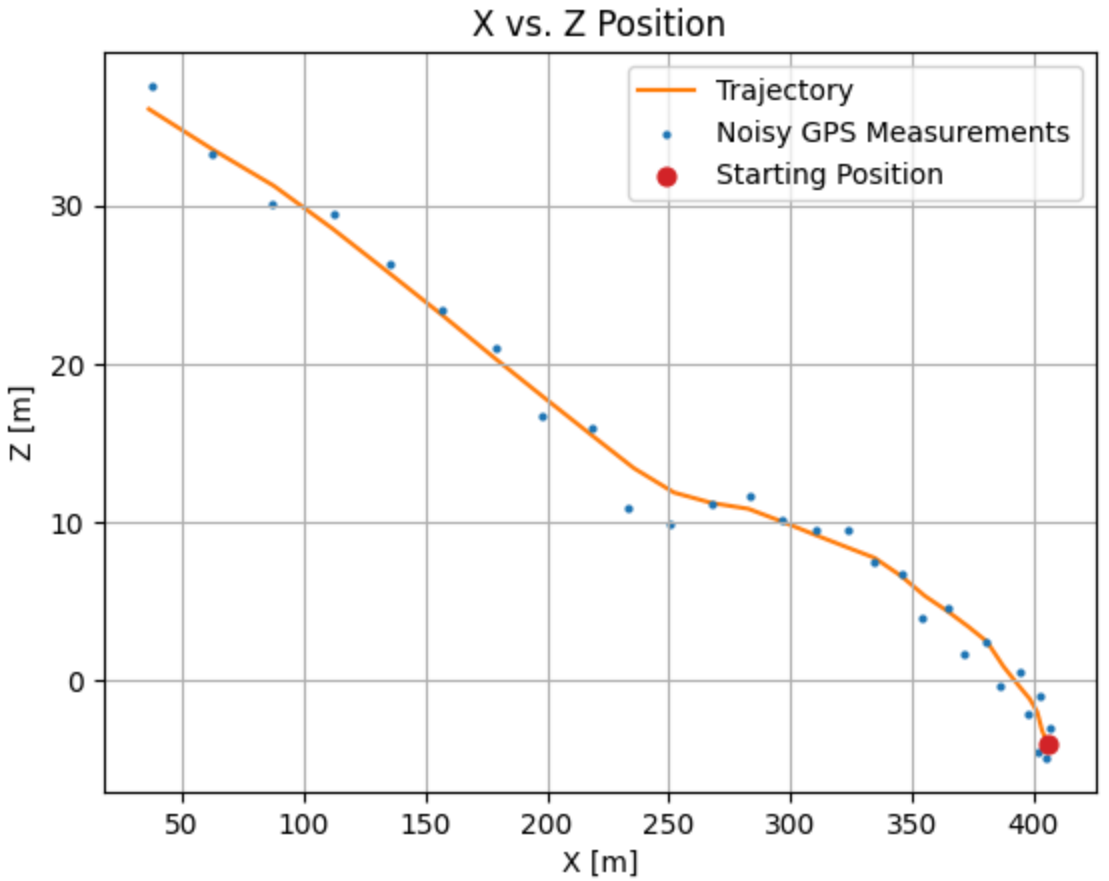
# traj = one_trajectory(30, Q, R)

# traj = one_trajectory(30, np.zeros((6,6)), np.ones((3,3)))
# traj = one_trajectory(30, np.zeros((6,6)), 0.1*np.ones((3,3)))
traj = one_trajectory(30, Q, np.ones((3,3)))
```

```
[[0.  0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.1 0.  0. ]
 [0.  0.  0.  0.  0.1 0. ]
 [0.  0.  0.  0.  0.  0.1]]
[[100.   0.   0.]
 [  0. 100.   0.]
 [  0.   0. 100.]]
[ 406.   -203.     -4.06    0.      0.      0.  ]
```

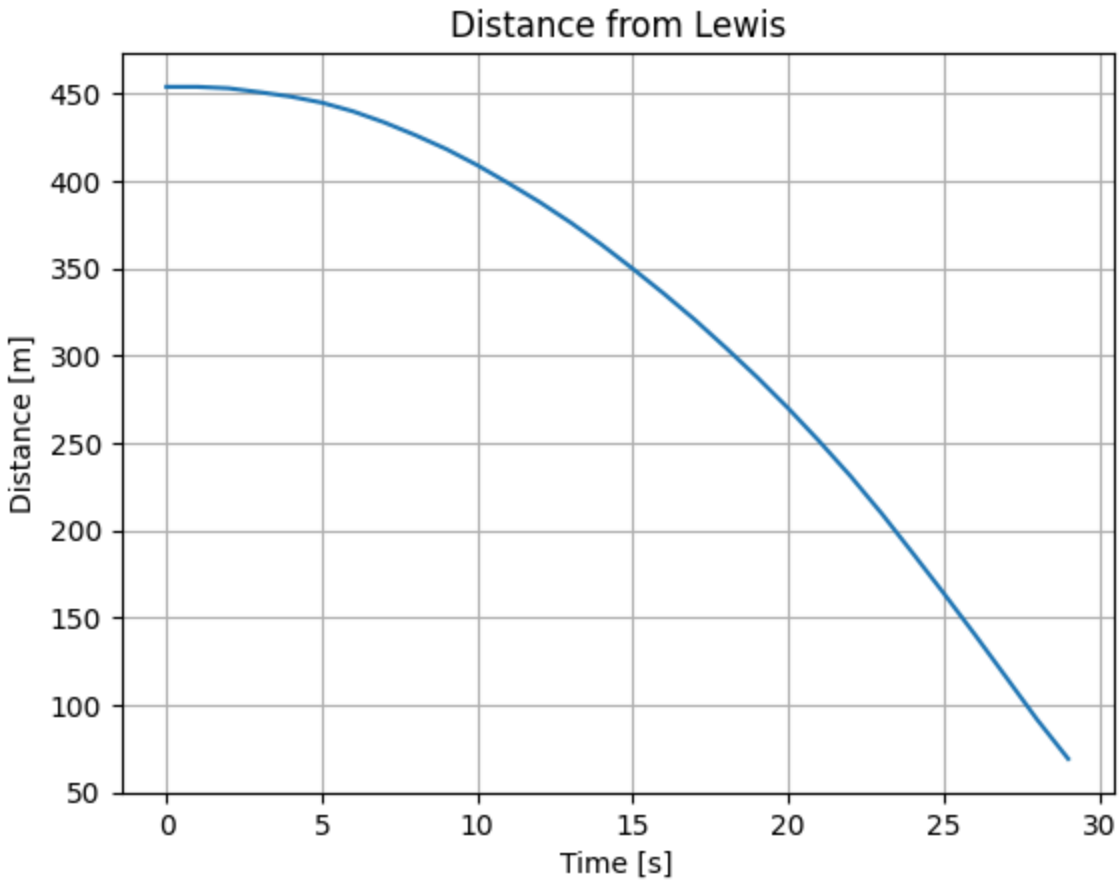



Y vs. Z Position
Trajectory
Noisy GPS Measurements
Starting Position
Z [m]
Y [m]
30
20
10
0
−200
−180
−160
−140
−120
−100
−80
−60
−40


X vs. Z Position
Trajectory
Noisy GPS Measurements
Starting Position
Z [m]
X [m]
30
20
10
0
50
100
150
200
250
300
350
400

In [119…
```
dist = []
for i in traj.x_arr:
    dist.append(np.linalg.norm(i[:3]))
dist = np.asarray(dist)
plt.plot(dist)
plt.grid()
plt.title("Distance from Lewis")
plt.xlabel("Time [s]")
plt.ylabel("Distance [m]")
```

Out[119…
```
Text(0, 0.5, 'Distance [m]')
```



Running Kalman Filter

In [119…
```
mu0 = np.array([406, -203, -4.06, 0, 0, 0])
Sigma0 = np.zeros((6,6))
Sigma0[0:3, 0:3] = 1000*np.eye(3)
Sigma0[3:6, 3:6] = np.eye(3)
```

In [119…
```
mu_arr = np.zeros((N,6))
Sigma_arr = np.zeros((N,6,6))

mu_arr[0] = mu0
Sigma_arr[0] = Sigma0

x_arr = traj.x_arr
u_arr = traj.u_arr
```

```
meas_arr = traj.meas_arr

np.random.seed(273)
for t in range(N-1):
    mu_predict = A @ mu_arr[t] + B @ u_arr[t]
    Sigma_predict = A @ Sigma_arr[t] @ A.T + Q

    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R)

    mu_arr[t+1] = mu_predict + K @ (meas_arr[t] - C @ mu_predict)
    Sigma_arr[t+1] = Sigma_predict - K @ C @ Sigma_predict
```

In [119…

```
dimA = 0
dimB = 1
plt.plot(x_arr[:,dimA], x_arr[:,dimB], color="coral", zorder=1)
plt.scatter([mu_arr[0,dimA]], [mu_arr[0,dimB]], color="tab:pink", s=20, mark

mu_dimA = mu_arr[:,dimA]
mu_dimB = mu_arr[:,dimB]
plt.plot(mu_dimA, mu_dimB, color='tab:blue')
# for mu, sigma in zip(mu_arr, Sigma_arr):
#     mu = mu[0:2]
#     sigma = sigma[0:2, 0:2]
#     # plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
#     plt.scatter([mu[dimA]], [mu[dimB]], color='tab:blue', s=1)
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.title("X vs Y Position")
plt.grid()
plt.legend(("Trajectory", "Starting Estimate", "Estimated Positions"))
plt.show()

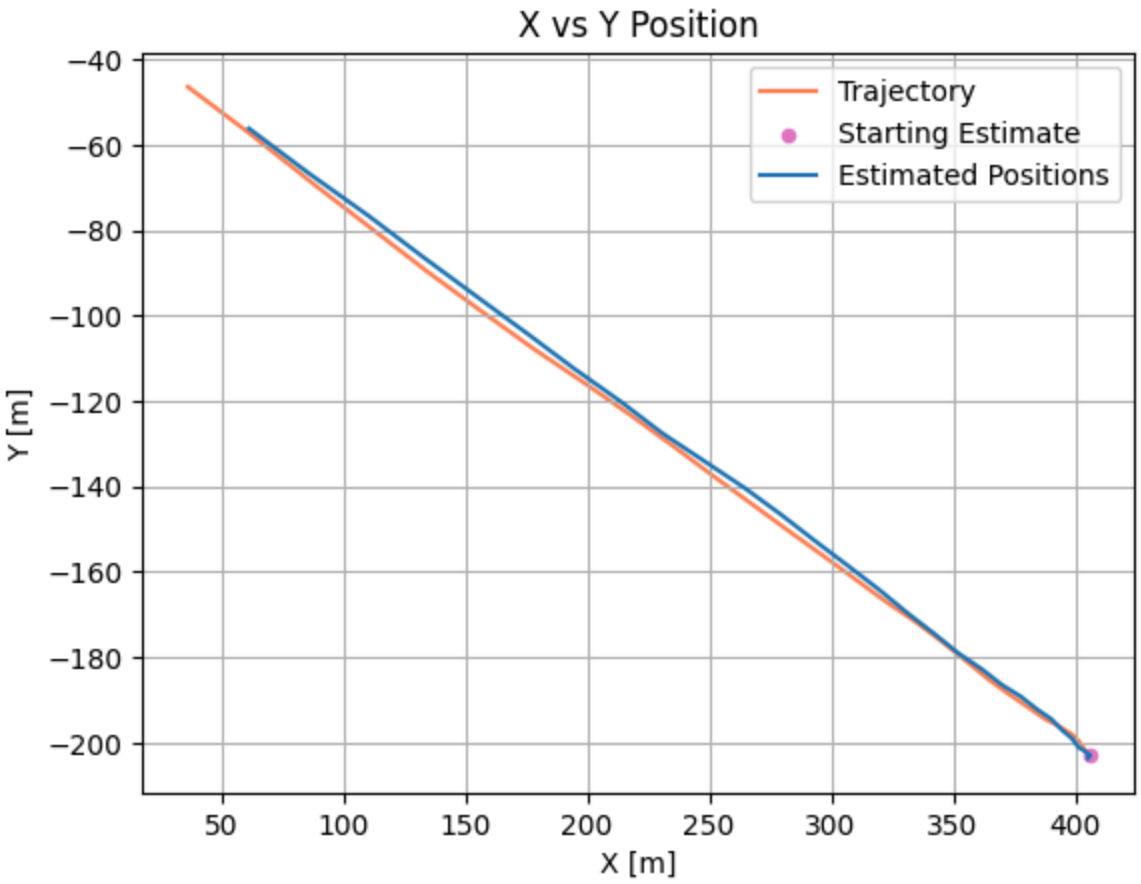
dimA = 0
dimB = 2
plt.plot(x_arr[:,dimA], x_arr[:,dimB], color="coral", zorder=1)
plt.scatter([mu_arr[0,dimA]], [mu_arr[0,dimB]], color="tab:pink", s=20, mark

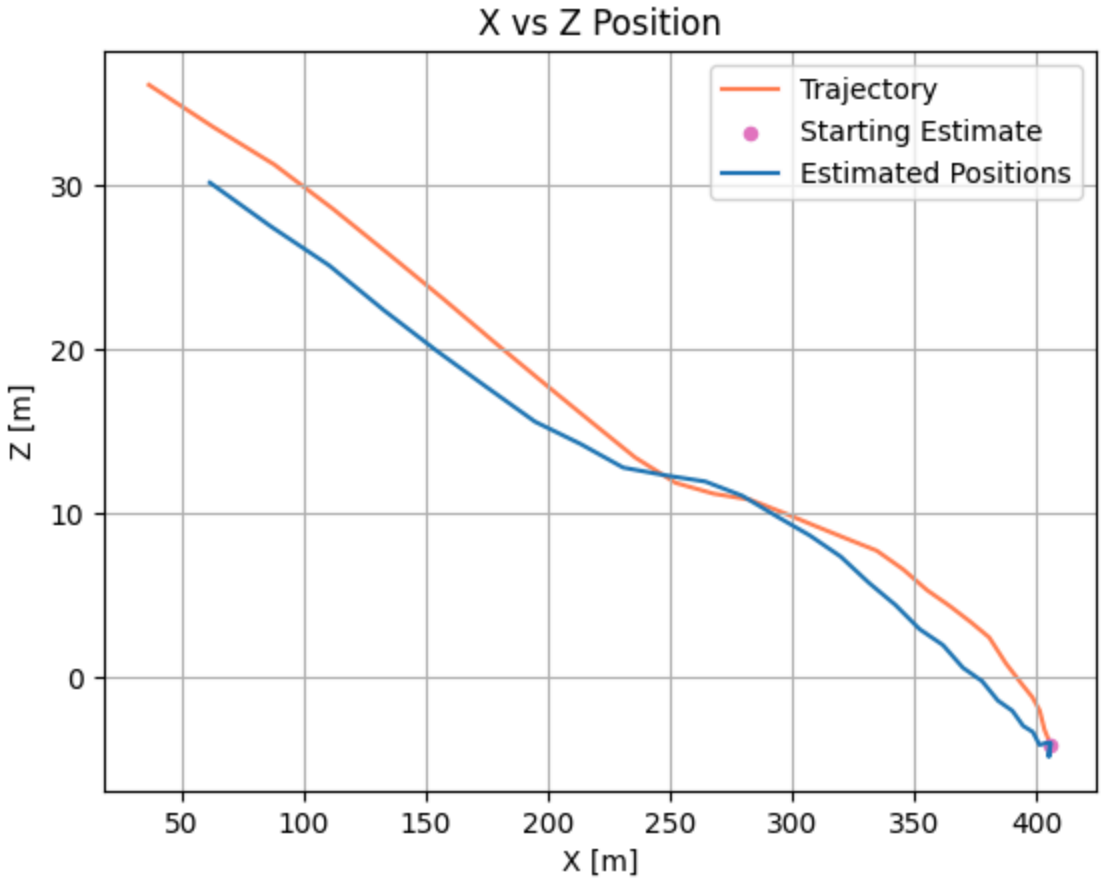
mu_dimA = mu_arr[:,dimA]
mu_dimB = mu_arr[:,dimB]
plt.plot(mu_dimA, mu_dimB, color='tab:blue')
# for mu, sigma in zip(mu_arr, Sigma_arr):
#     sigma = sigma[0:2, 0:2]
#     # plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
#     plt.scatter([mu[dimA]], [mu[dimB]], color='tab:blue', s=1)
plt.xlabel("X [m]")
plt.ylabel("Z [m]")
plt.title("X vs Z Position")
plt.grid()
plt.legend(("Trajectory", "Starting Estimate", "Estimated Positions"))

plt.show()

dimA = 1
dimB = 2
plt.plot(x_arr[:,dimA], x_arr[:,dimB], color="coral", zorder=1)
plt.scatter([mu_arr[0,dimA]], [mu_arr[0,dimB]], color="tab:pink", s=20, mark
```

```
mu_dimA = mu_arr[:,dimA]
mu_dimB = mu_arr[:,dimB]
plt.plot(mu_dimA, mu_dimB, color='tab:blue')
# for mu, sigma in zip(mu_arr, Sigma_arr):
#     # mu = mu[0:2]
#     sigma = sigma[0:2, 0:2]
#     # plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
#     plt.scatter([mu[dimA]], [mu[dimB]], color='tab:blue', s=1)
plt.xlabel("Y [m]")
plt.ylabel("Z [m]")
plt.title("Y vs Z Position")
plt.grid()
plt.legend(("Trajectory", "Starting Estimate", "Estimated Positions"))
plt.show()
```







In [119…
```
print(Sigma_arr[-1])
```

```
[[22.2689  0.      0.      2.7869  0.      0.    ]
 [ 0.     22.2689  0.      0.      2.7869  0.    ]
 [ 0.      0.     22.2689  0.      0.      2.7869]
 [ 2.7869  0.      0.      0.7991  0.      0.    ]
 [ 0.      2.7869  0.      0.      0.7991  0.    ]
 [ 0.      0.      2.7869  0.      0.      0.7991]]
```

In [119…

```
mu0_new = np.array([406, -203, -4.06, 0, 0, 0])
# Sigma0_new = np.zeros((6,6))
# Sigma0_new[0:3, 0:3] = 1000*np.eye(3)
# Sigma0_new[3:6, 3:6] = np.eye(3)

Sigma0_new = Sigma0*10
# Sigma0_new = Sigma0
```

In [119…

```
# PART C
#
# Q_new = Q
Q_new = 10*Q
R_new = 10*R
# R_new = 0.1*np.eye(3)

print(Q_new)
print(R_new)
```

```
[[0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 1.]]
[[1000.    0.    0.]
 [   0. 1000.    0.]
 [   0.    0. 1000.]]
```

In [119…

```
mu_arr = np.zeros((N,6))
Sigma_arr = np.zeros((N,6,6))

mu_arr[0] = mu0
Sigma_arr[0] = Sigma0_new

x_arr = traj.x_arr
u_arr = traj.u_arr
meas_arr = traj.meas_arr


for t in range(N-1):
    mu_predict = A @ mu_arr[t] + B @ u_arr[t]
    Sigma_predict = A @ Sigma_arr[t] @ A.T + Q_new

    K = Sigma_predict @ C.T @ inv(C @ Sigma_predict @ C.T + R_new)

    mu_arr[t+1] = mu_predict + K @ (meas_arr[t] - C @ mu_predict)
    Sigma_arr[t+1] = Sigma_predict - K @ C @ Sigma_predict
```

In [120…

```
dimA = 0
dimB = 1
plt.plot(x_arr[:,dimA], x_arr[:,dimB], color="coral", zorder=1)
plt.scatter([mu_arr[0,dimA]], [mu_arr[0,dimB]], color="tab:pink", s=20, mark

mu_dimA = mu_arr[:,dimA]
mu_dimB = mu_arr[:,dimB]
plt.plot(mu_dimA, mu_dimB, color='tab:blue')
# for mu, sigma in zip(mu_arr, Sigma_arr):
#     mu = mu[0:2]
#     sigma = sigma[0:2, 0:2]
#     # plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
#     plt.scatter([mu[dimA]], [mu[dimB]], color='tab:blue', s=1)
plt.xlabel("X [m]")
plt.ylabel("Y [m]")
plt.title("X vs Y Position")
plt.grid()
plt.legend(("Trajectory", "Starting Estimate", "Estimated Positions"))
plt.show()

dimA = 0
dimB = 2
plt.plot(x_arr[:,dimA], x_arr[:,dimB], color="coral", zorder=1)
plt.scatter([mu_arr[0,dimA]], [mu_arr[0,dimB]], color="tab:pink", s=20, mark

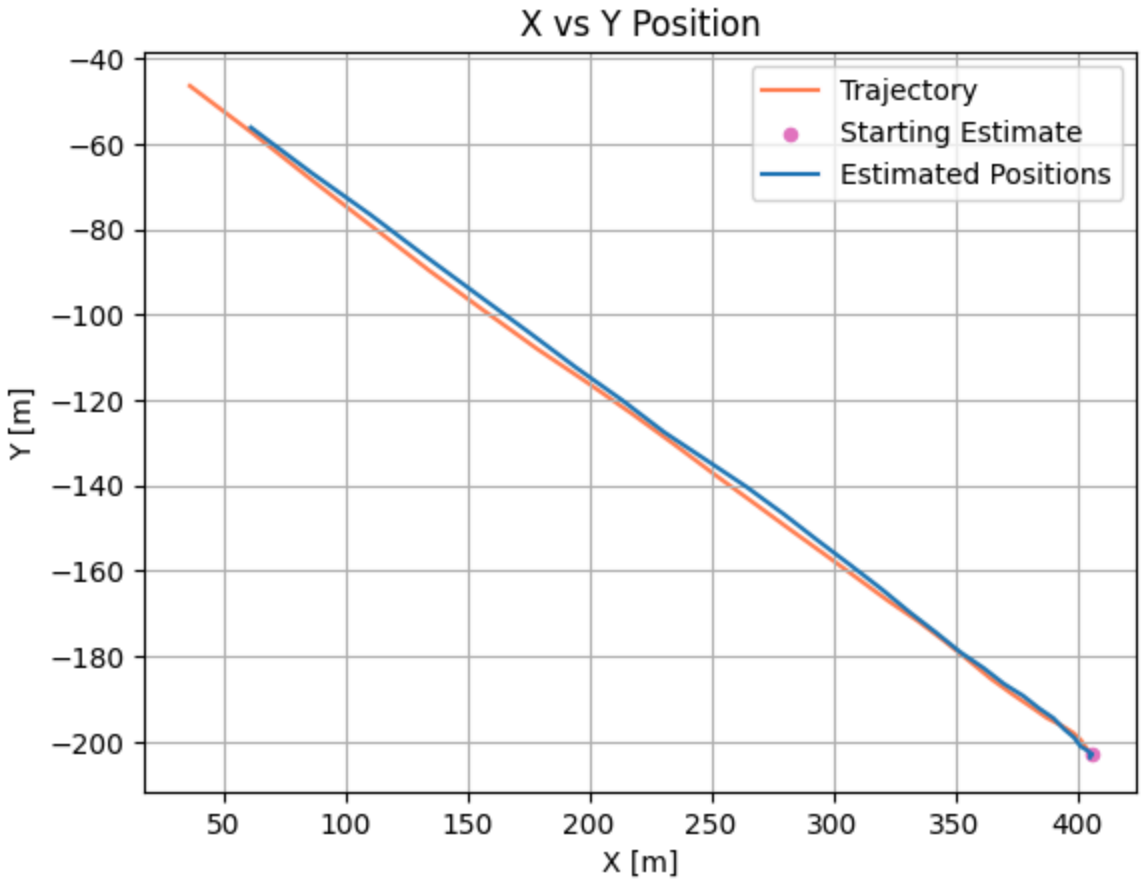
mu_dimA = mu_arr[:,dimA]
mu_dimB = mu_arr[:,dimB]
plt.plot(mu_dimA, mu_dimB, color='tab:blue')
# for mu, sigma in zip(mu_arr, Sigma_arr):
#     sigma = sigma[0:2, 0:2]
#     # plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
#     plt.scatter([mu[dimA]], [mu[dimB]], color='tab:blue', s=1)
plt.xlabel("X [m]")
plt.ylabel("Z [m]")
plt.title("X vs Z Position")
plt.grid()
plt.legend(("Trajectory", "Starting Estimate", "Estimated Positions"))

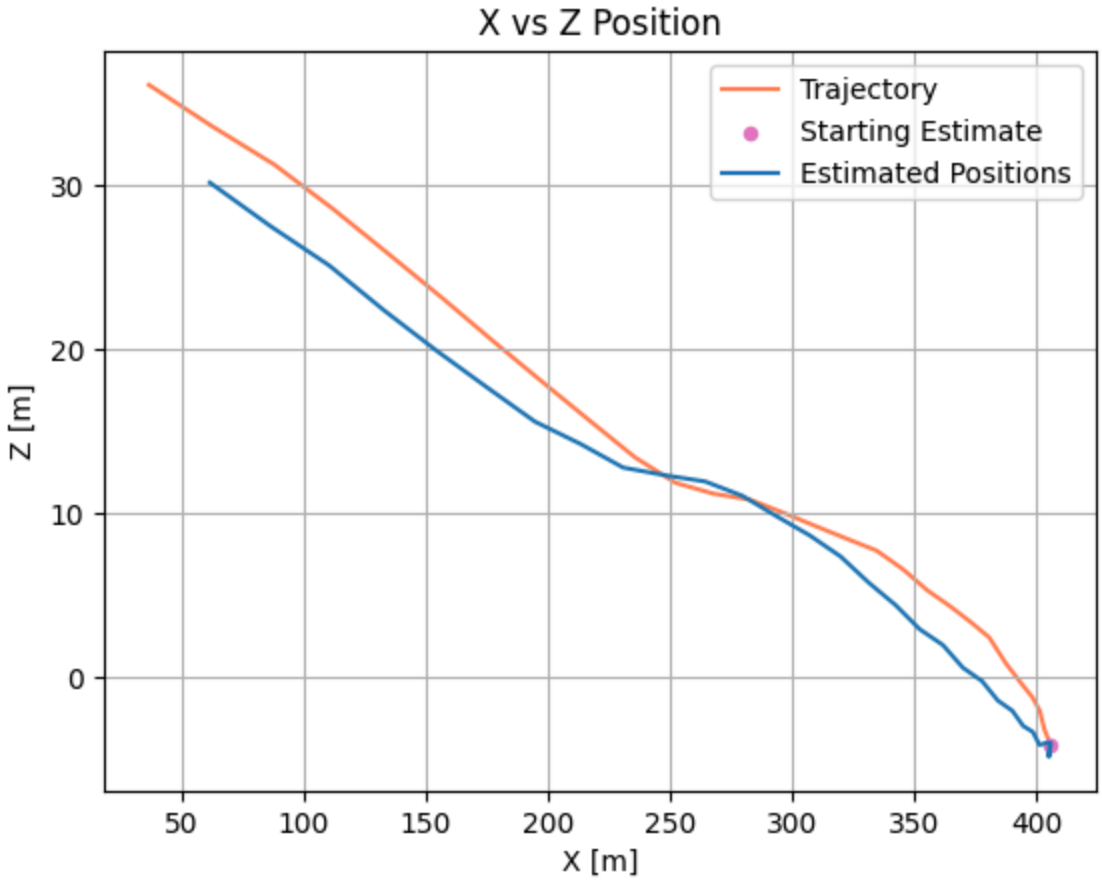
plt.show()

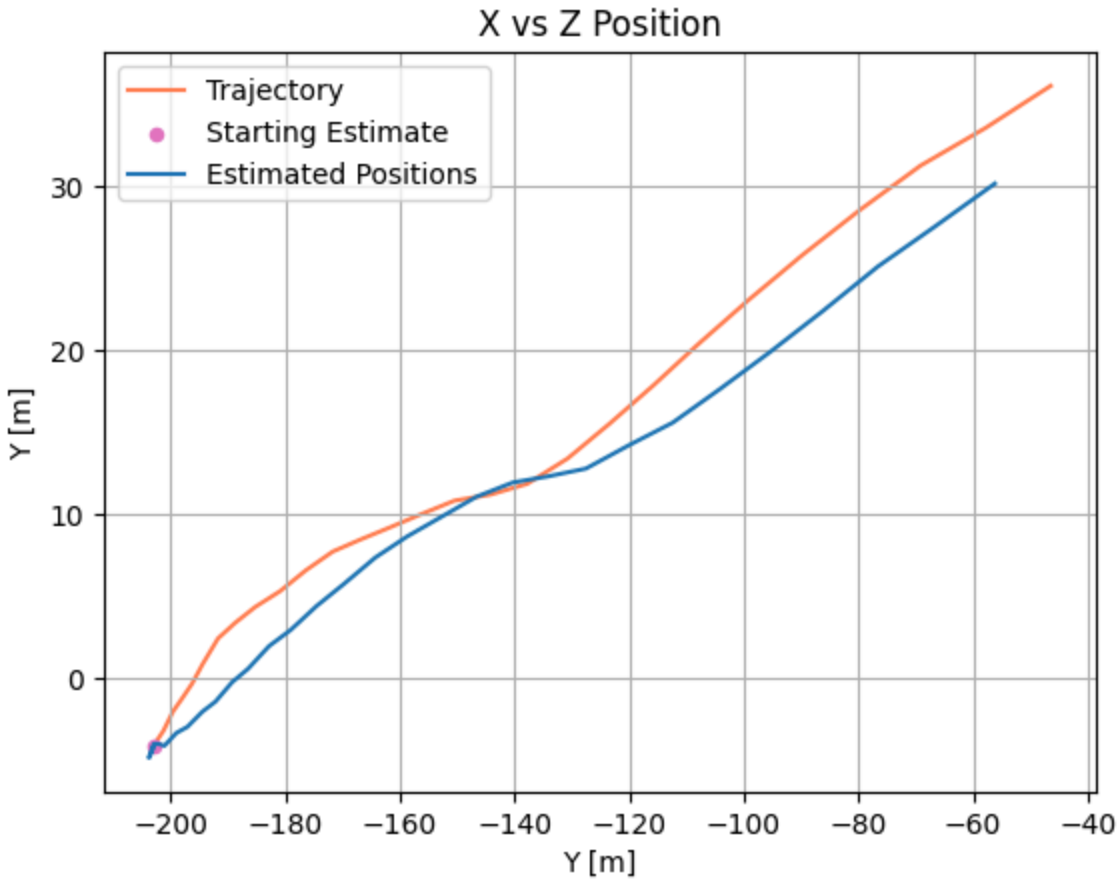
dimA = 1
dimB = 2
plt.plot(x_arr[:,dimA], x_arr[:,dimB], color="coral", zorder=1)
plt.scatter([mu_arr[0,dimA]], [mu_arr[0,dimB]], color="tab:pink", s=20, mark

mu_dimA = mu_arr[:,dimA]
mu_dimB = mu_arr[:,dimB]
plt.plot(mu_dimA, mu_dimB, color='tab:blue')
# for mu, sigma in zip(mu_arr, Sigma_arr):
#     # mu = mu[0:2]
#     sigma = sigma[0:2, 0:2]
#     # plt.plot(*get_ellipse_points(mu, sigma), color='tab:blue')
#     plt.scatter([mu[dimA]], [mu[dimB]], color='tab:blue', s=1)
plt.xlabel("Y [m]")
plt.ylabel("Y [m]")
```

```
plt.title("X vs Z Position")
plt.grid()
plt.legend(("Trajectory", "Starting Estimate", "Estimated Positions"))
plt.show()
```




X vs Z Position
Trajectory
Starting Estimate
Estimated Positions
Z [m]
X [m]
30
20
10
0
50
100
150
200
250
300
350
400


X vs Z Position
Trajectory
Starting Estimate
Estimated Positions
Y [m]
Y [m]
30
20
10
0
−200
−180
−160
−140
−120
−100
−80
−60
−40

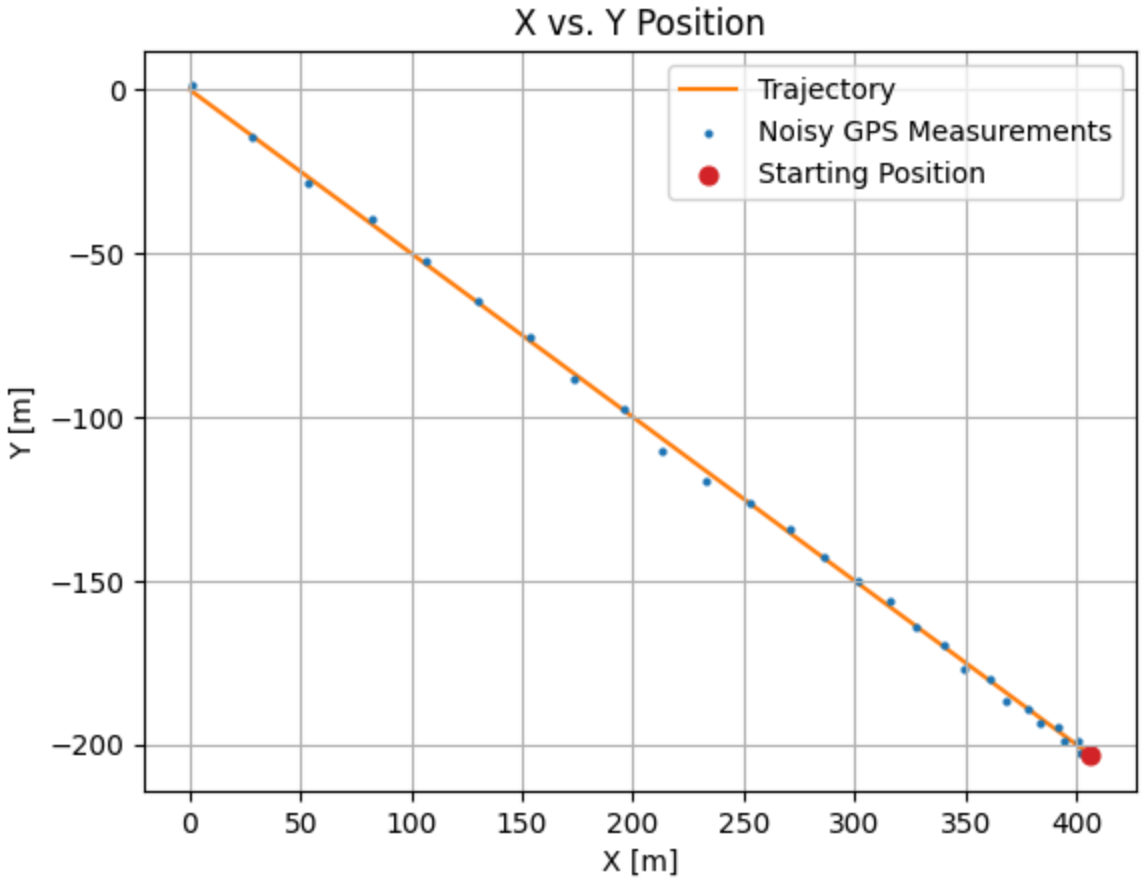
In [120…
```
np.set_printoptions(precision=4)
print(Sigma_arr[-1])
```

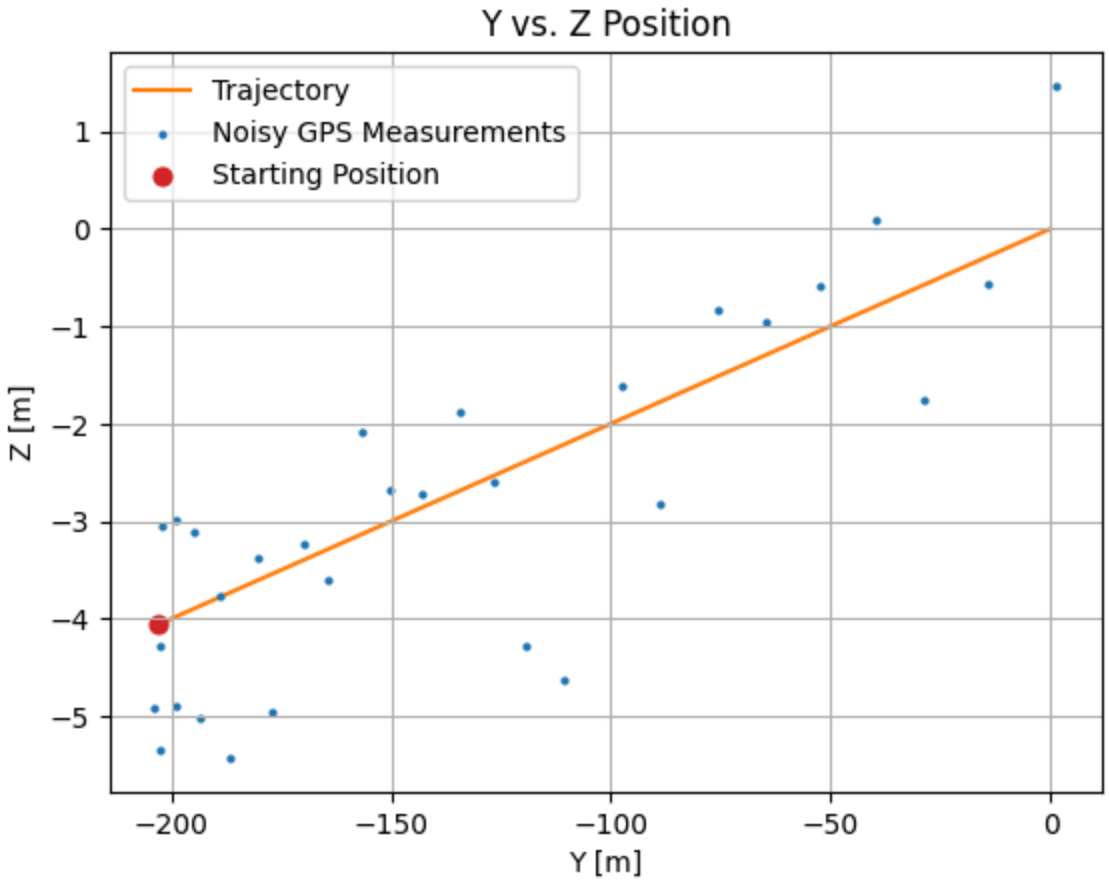
```
[[222.6891   0.       0.      27.8687   0.       0.    ]
 [  0.     222.6891   0.       0.      27.8687   0.    ]
 [  0.       0.     222.6891   0.       0.      27.8687]
 [ 27.8687   0.       0.       7.9914   0.       0.    ]
 [  0.      27.8687   0.       0.       7.9914   0.    ]
 [  0.       0.      27.8687   0.       0.       7.9914]]
```

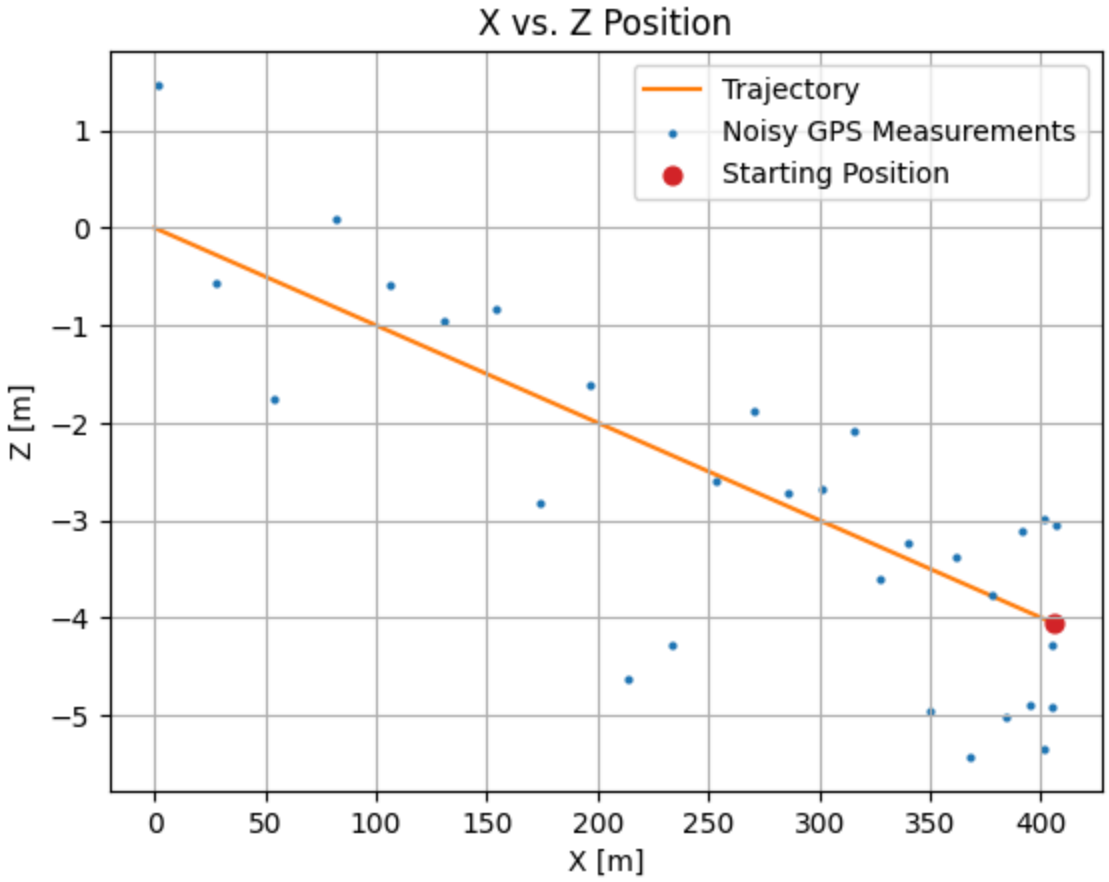
In [120…
```
# PART C
#

Q_new = Q_OG
# print(Q_OG)
# R_new = 10*R
R_new = 0.1*np.eye(3)
# print(Q_new)
# print(Q_OG)
# print(R_OG)
# print(R_new)
print(Q_new)
# R_OG
np.random.seed(273)
traj = one_trajectory(30, np.zeros((6,6)), np.ones((3,3)))
# traj = one_trajectory(30, np.zeros((6,6)), 0.1*np.ones((3,3)))
# traj = one_trajectory(30, Q_new, np.ones((3,3)))
```

```
[[0.  0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.1 0.  0. ]
 [0.  0.  0.  0.  0.1 0. ]
 [0.  0.  0.  0.  0.  0.1]]
[ 406.   -203.     -4.06    0.      0.      0.  ]
```


X vs. Y Position
Trajectory
Noisy GPS Measurements
Starting Position
Y [m]
X [m]
0
−50
−100
−150
−200
0
50
100
150
200
250
300
350
400


Y vs. Z Position
Trajectory
Noisy GPS Measurements
Starting Position
Z [m]
Y [m]
1
0
−1
−2
−3
−4
−5
−200
−150
−100
−50
0


X vs. Z Position
Trajectory
Noisy GPS Measurements
Starting Position
Z [m]
X [m]
1
0
−1
−2
−3
−4
−5
0
50
100
150
200
250
300
350
400